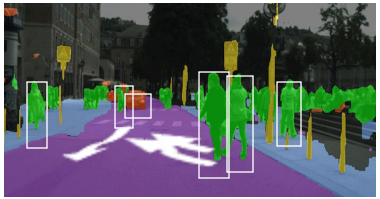
Deep Learning for Autonomous Vehicles

Lecture 2:
Perception

A. Alahi

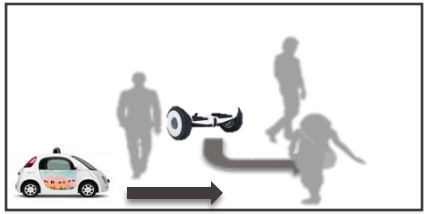
Last week



Social Forecasting

Perceiving

Socially-aware AI



Planning

Last week

- Sensing
- Image classification
- Quick recap of Machine Learning (ML)
  - Nearest Neighbor
  - Linear classifier
  - Loss functions
  - Optimization

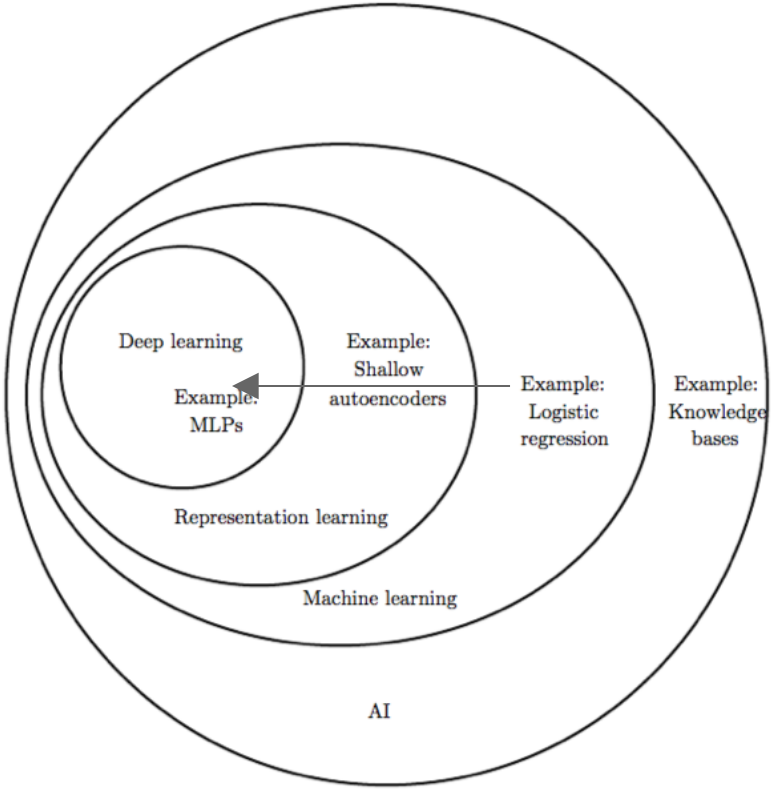
So far

- Dataset of (x,y)
- Score function, e.g., linear classifier f(x,W) =Wx+b
- Loss functions e.g., softmax loss (quantifies how good is W)
- Optimization methods (find the best W)

Today

- New functional for the score function f(x,W)
  - Changing representation x
  - Learning representation
  - Neural Network
  - Convolutional Neural Networks
  - Regularization

Where are we going

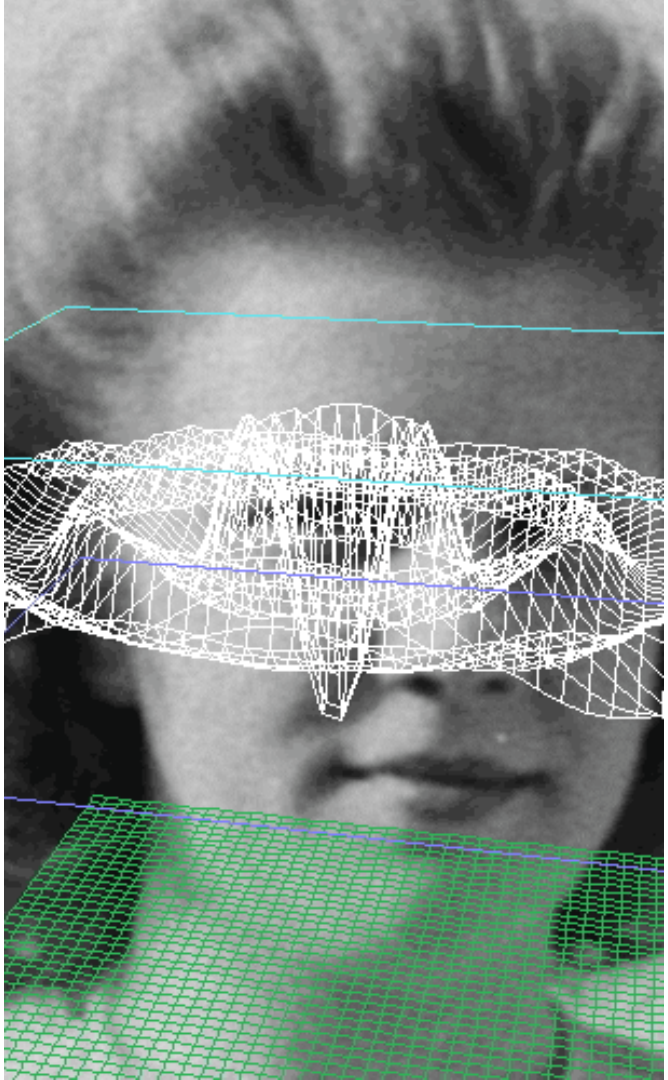


Now

- Representation learning

Role of representation in ML

- A representation is a mathematical form (e.g., a vector)
- It describes an observation in the real-world (e.g., an image, waveforms, signals)
- It is often more compact than the original observation (i.e., lower dimensionality)
- It is potentially more robust to nuisances (e.g., illumination changes in images)
- It is used to ease subsequent steps (e.g., a classifier)



Role of representation in ML

- Forming the proper representation for a task is an **essential** problem in AI/Perception/Computer Vision

- *"Solving a problem simply means representing it so as to make the solution transparent."*, H. Simon, Sciences of the Artificial

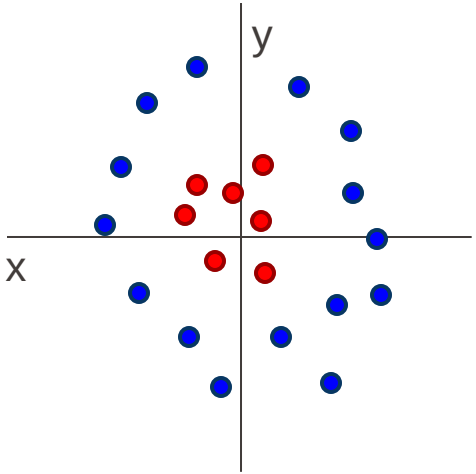
Role of representation in ML

Ex: An AI system needs to recommend a surgery:

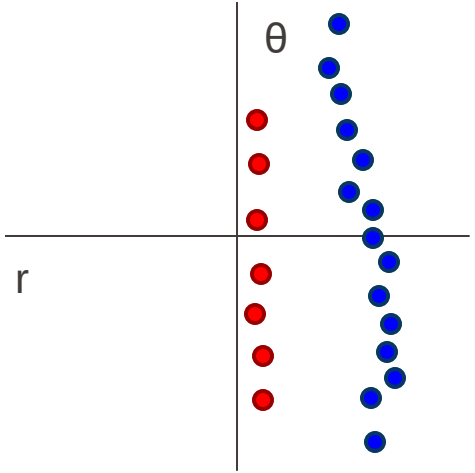
- AI does not examine the patient directly
- Instead, the doctor tells **key** information
- Each information is included in the representation of the patient (aka *feature*)
- Logistic regression can learn how the features correlates with various outcomes

- Now suppose, we use as feature each pixel of an MRI scan (rather than the doctor's key information)
- Logistic regression will fail to make good prediction since individual pixels in an MRI have negligible correlation with the target outcome

Power of representation



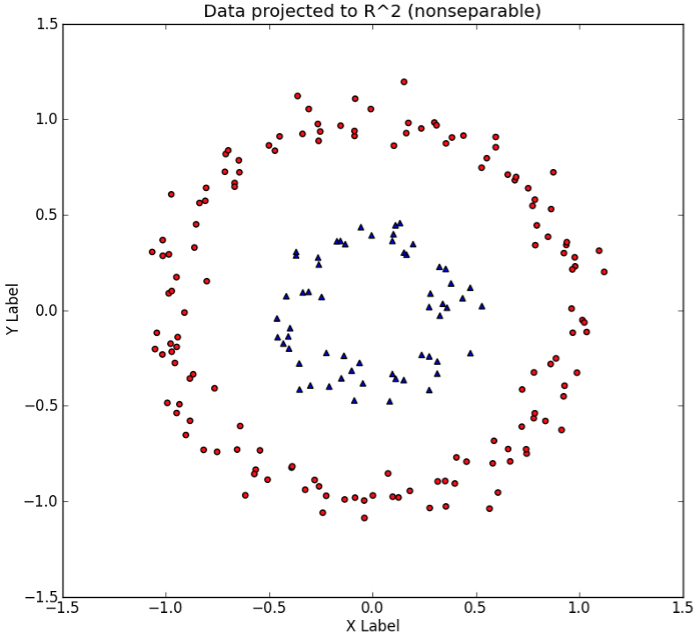
$$f(x, y) = (r(x, y), \theta(x, y))$$

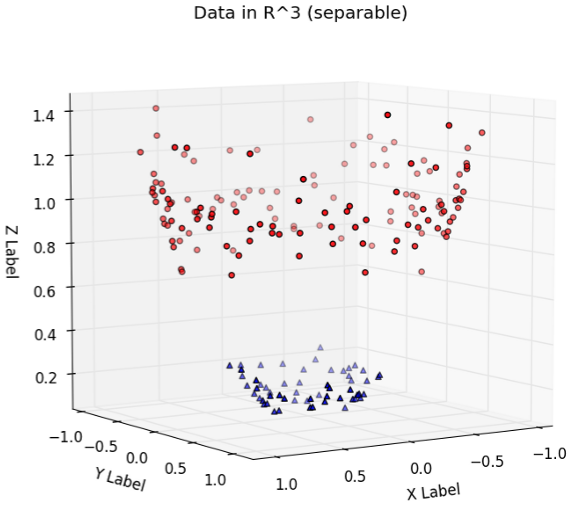


Cannot separate red and blue points with linear classifier

After applying feature transform, points can be separated by linear classifier

Power of representation





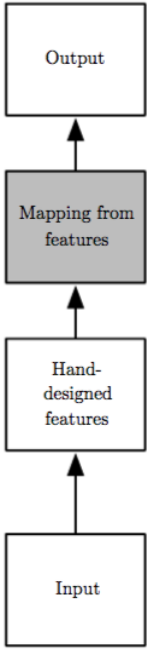
Source: towardsdatascience.com

First strategy: hand-designed representation

- Goal:

we want a representation that is:

- Descriptive enough to disambiguate objects VS background
- Flexible enough to cope with all nuisances (see lecture 2):
  - Scale
  - Pose
  - Illumination
  - Partial occlusions



Classic machine learning

Object representation: From light to useful information

Image

Array of **32x32x3** numbers
(3072 numbers total)

3072x1

$f(x,W) = Wx + b$

10x1

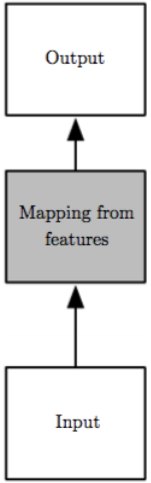
10x1

10x3072

$f(\mathbf{x},\mathbf{W})$

W
parameters
or weights

10 class scores



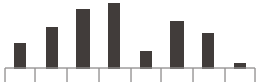
Classic machine learning

A. Alahi

Object representation: From light to useful information

Image

Array of **32x32x3** numbers
(3072 numbers total)



New representation:
Array of **8** numbers

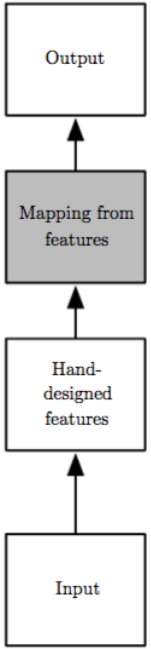
8x1

$$\underbrace{f(x,W)}_{10\text{x}1} = \underbrace{W}_{10\text{x}8}\,r + \underbrace{b}_{10\text{x}1}$$

$f(\mathbf{r},\mathbf{W})$

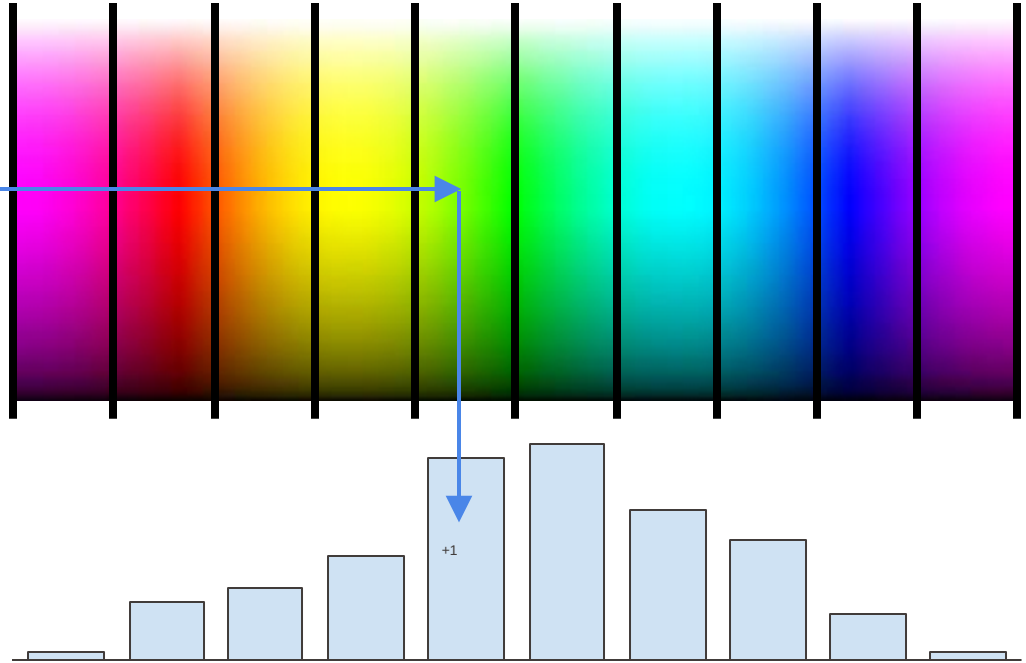
W
parameters
or weights

10 class scores

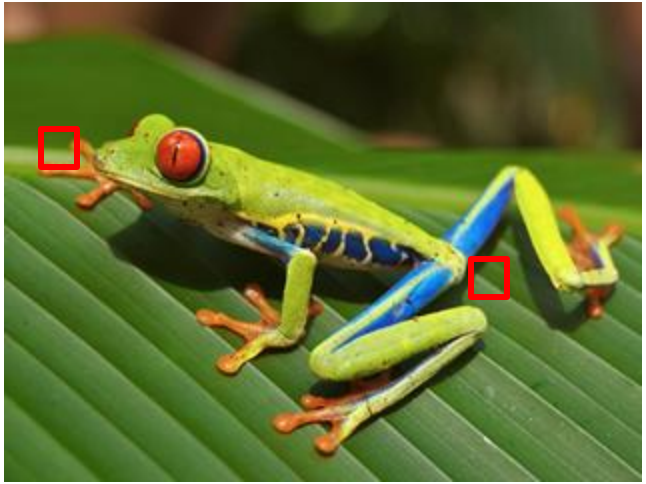


Classic machine learning

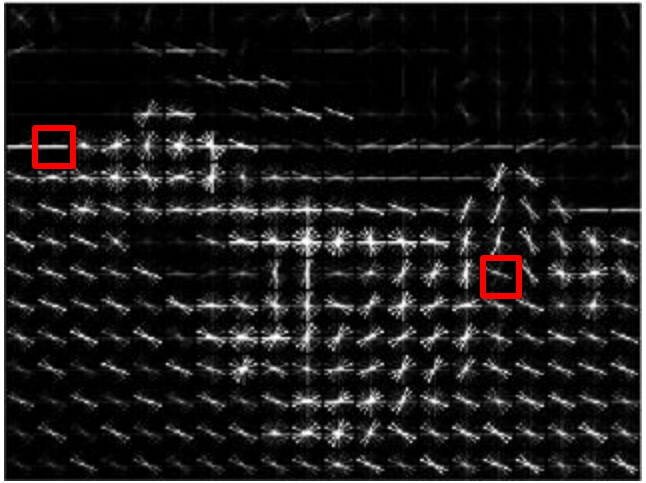
Example of representation: Color Histogram



Example of representation: Histogram of Oriented Gradients (HoG)



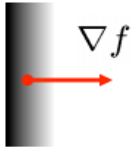
Divide image into 8x8 pixel regions
Within each region quantize edge direction into 9 bins



Example: 320x240 image gets divided into 40x30 bins; in each bin there are 9 numbers so feature vector has 30*40*9 = 10,800 numbers

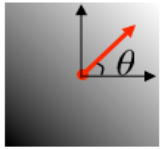
Image gradient

- The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$



$\nabla f = \left[\frac{\partial f}{\partial x}, 0\right]$

$\nabla f = \left[0, \frac{\partial f}{\partial y}\right]$



$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$

The gradient points in the direction of most rapid increase in intensity

The gradient direction is given by $\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$

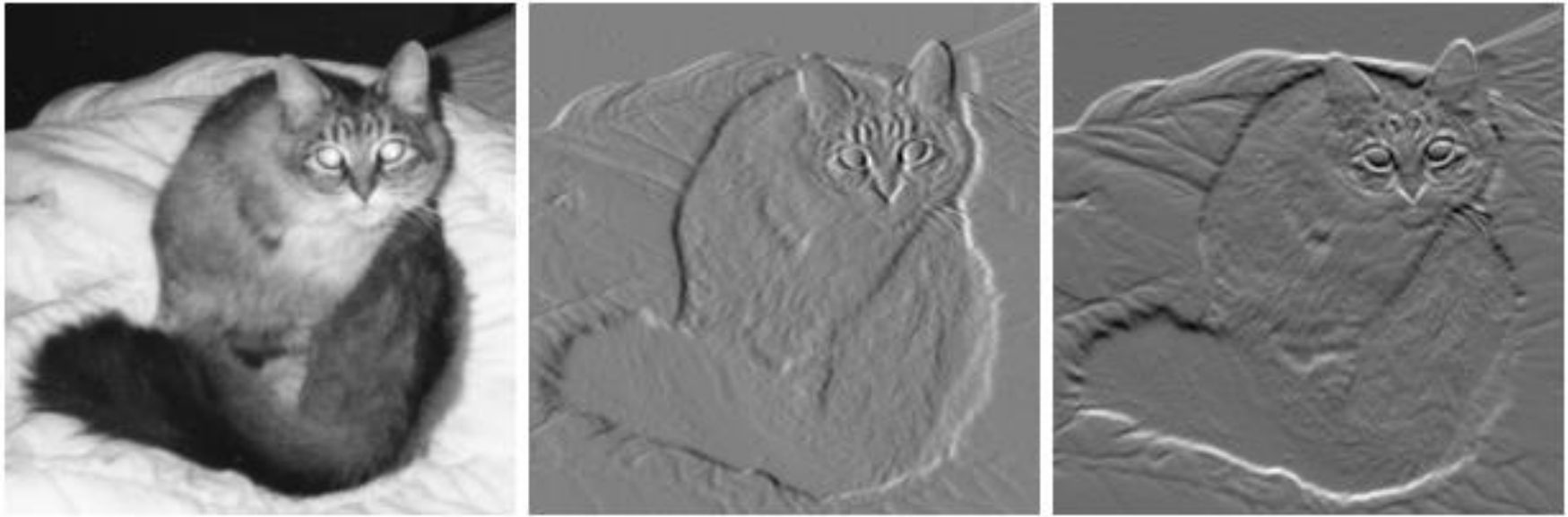
- how does this relate to the direction of the edge?

The *edge strength* is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Source: Steve Seitz

Image gradient

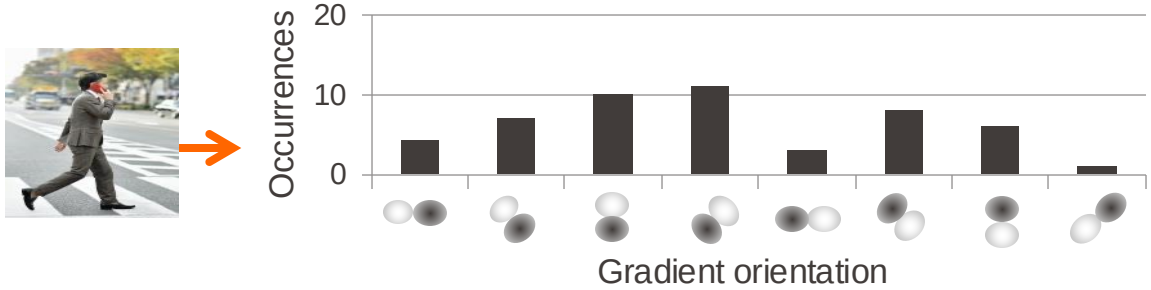


A. Alahi

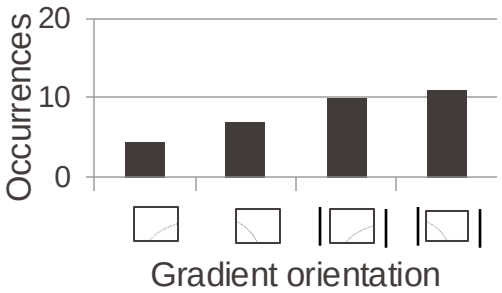
Low-level features

A. Alahi

- Integer responses

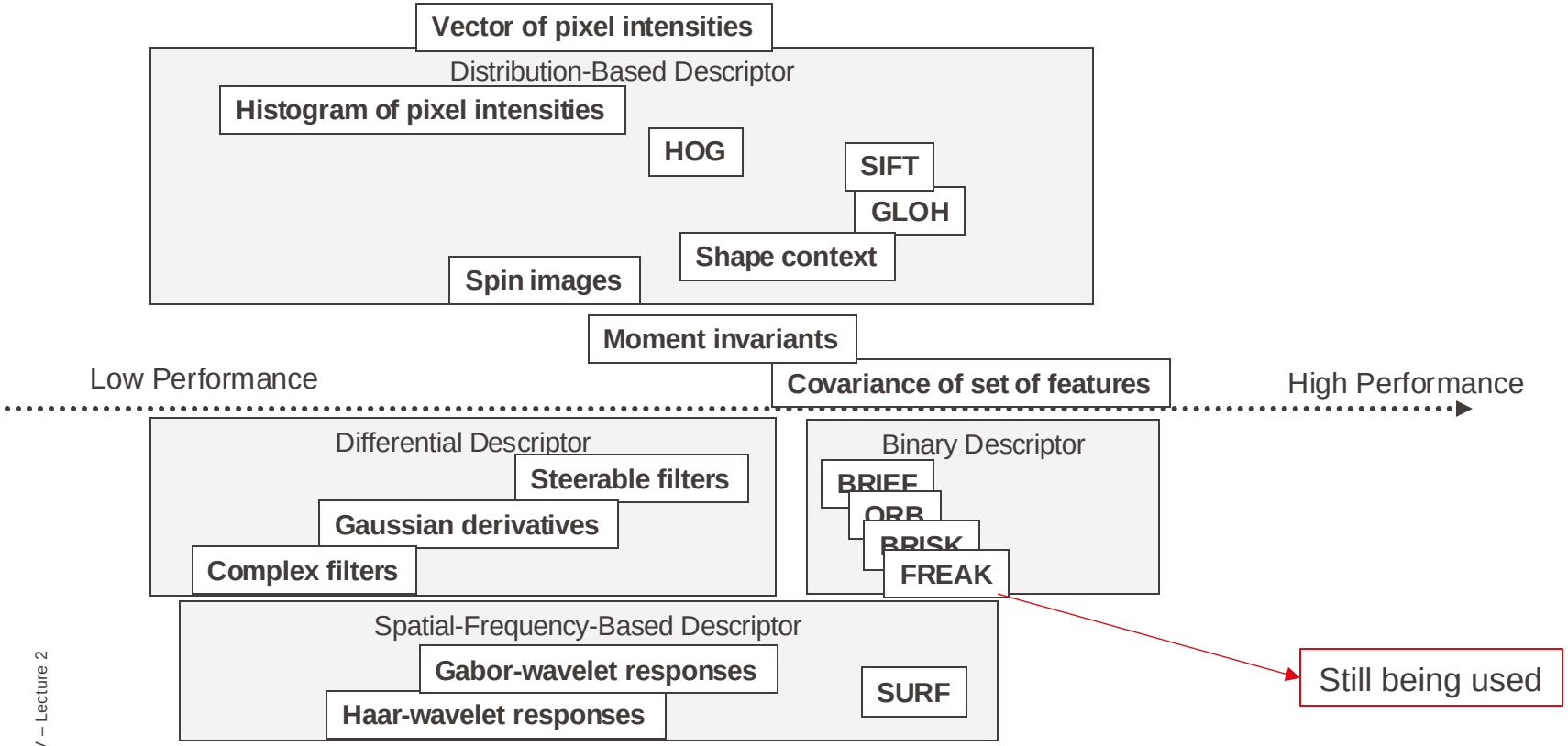


HoG feature used in SIFT-like descriptor



Haar feature used in SURF

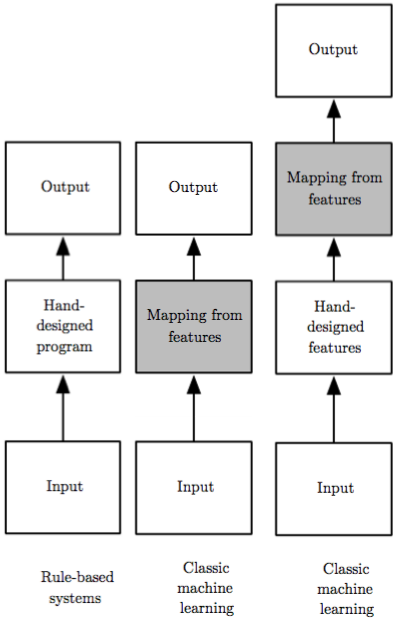
A bulk of Low-level features (pre deep learning era)



So far…

We saw:

- How to learn the mapping from an **input** (a representation) to an **output** (with optimization methods…)
- Impact of input representations
- Some hand-designed representations to improve performance



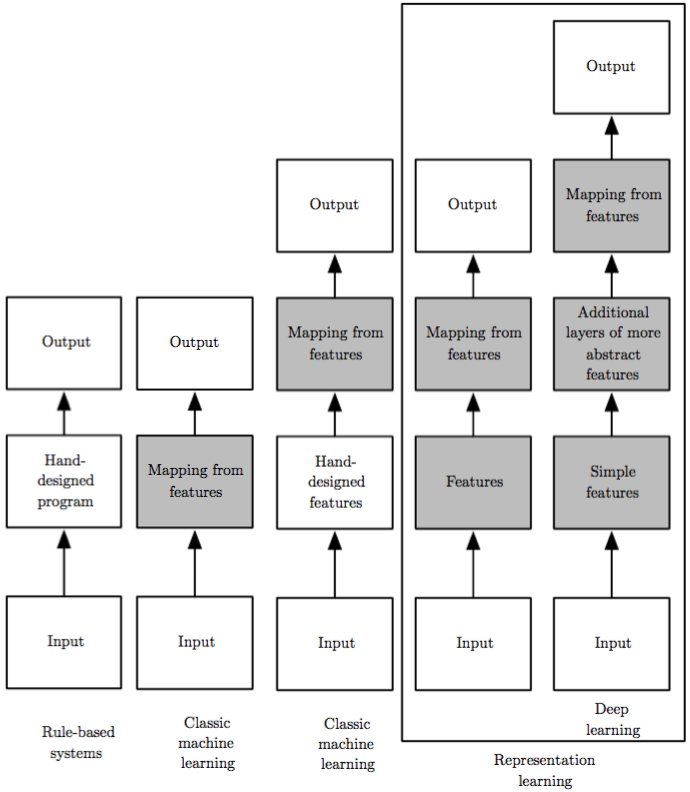
For pre-DL CV

Now…

Now, we will

- Learn how to also discover the representation itself, referred to **representation learning from the data**

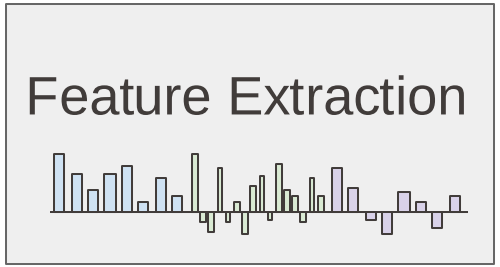
- Current state-of-the-art AI systems use learned representation *VS* hand-designed feature



For pre-DL CV

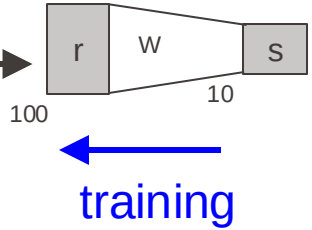
Image features

x



r

$$F = Wr = s$$

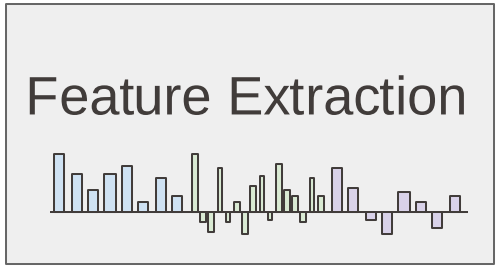


10 scores
for classes

Image features vs Neural networks

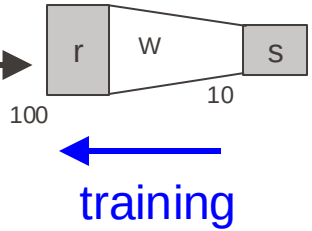
A. Alahi

x



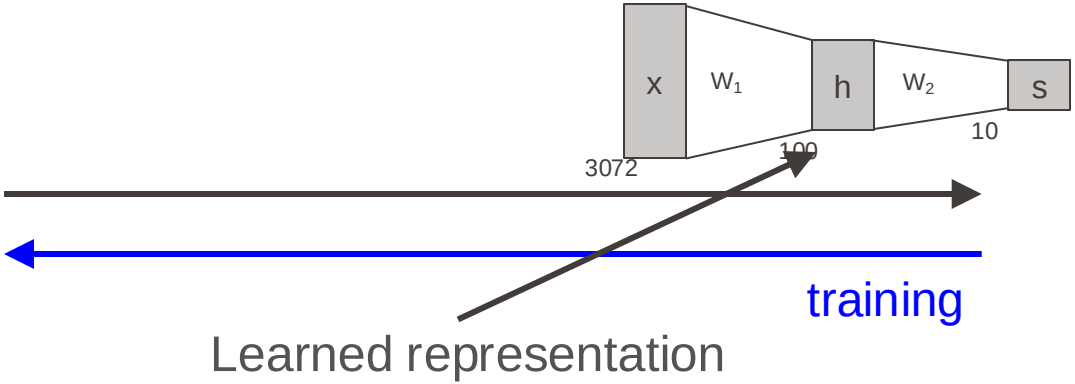
r

$F = Wr = s$



10 scores for classes

2-layer NN: $F = W_2 \max(0, W_1 x)$

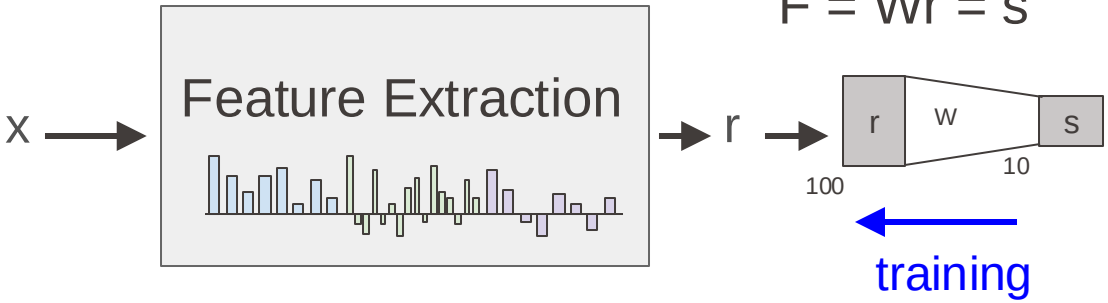


10 scores for classes

Image features vs Neural networks

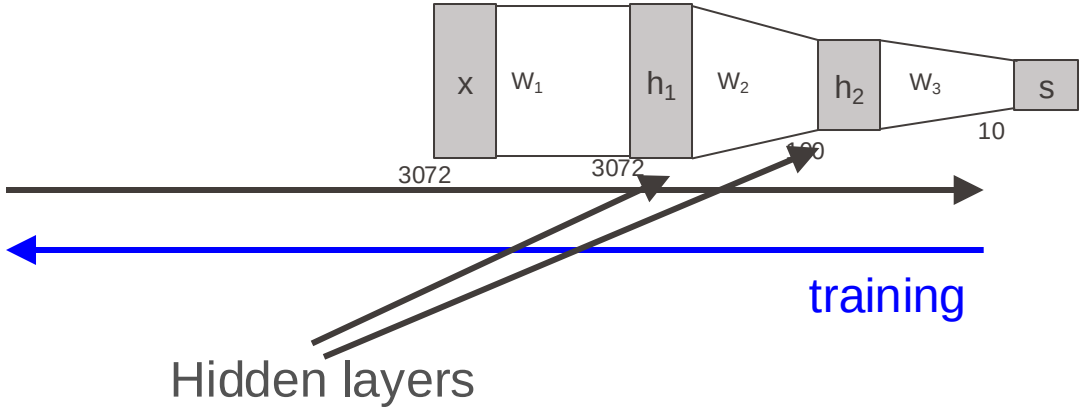
A. Alahi

$F = Wr = s$



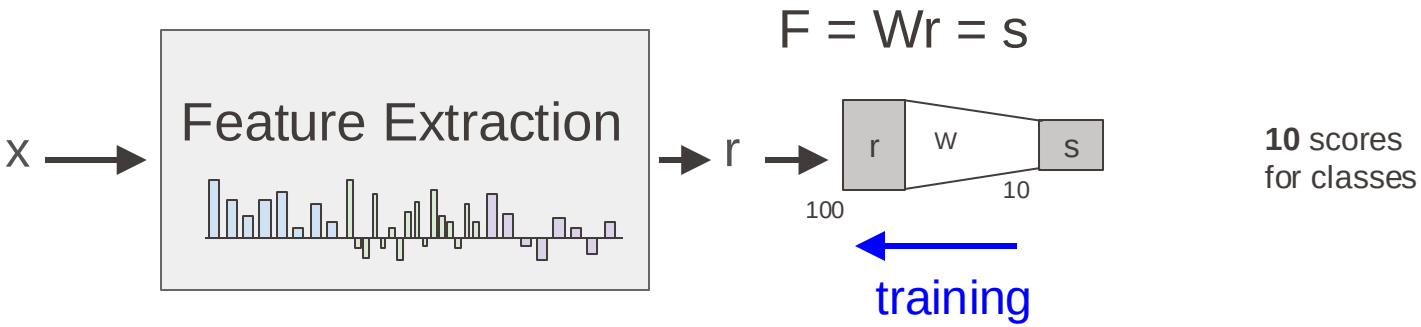
10 scores for classes

3-layer NN: $F = W_3 \max(0, W_2 \max(0, W_1 x))$



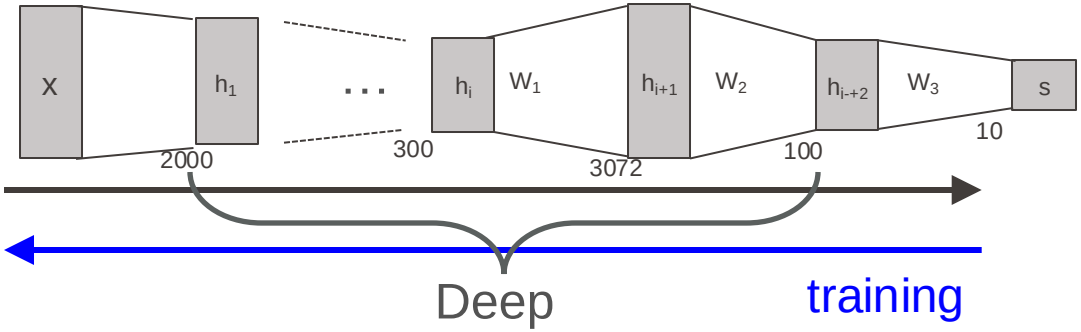
10 scores for classes

Image features vs Neural networks



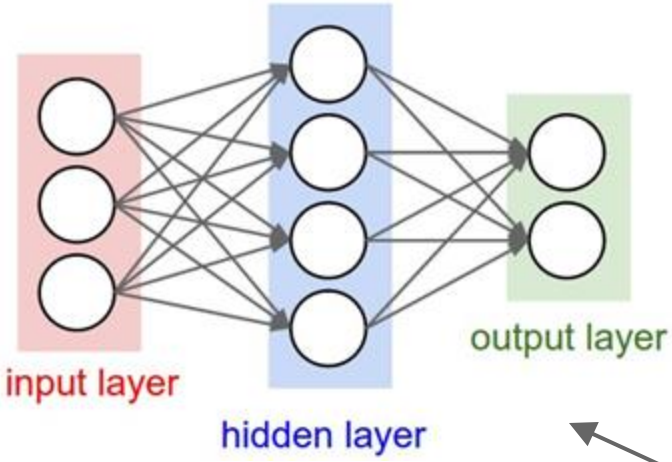
10 scores for classes

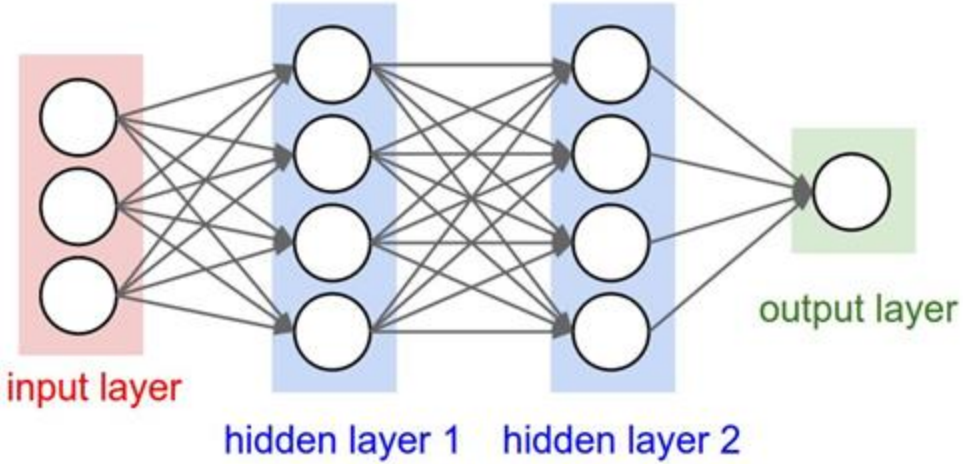
Deep representation learning (many layers)



10 scores for classes

Neural networks: Architectures





"2-layer Neural Net", or
"1-hidden-layer Neural Net"

"3-layer Neural Net", or
"2-hidden-layer Neural Net"

"Fully-connected" layers

A. Alahi

Image features vs Neural networks

Feature Extraction

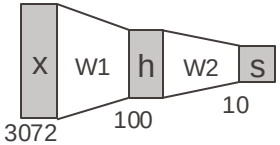
$F = Wx$

10 numbers giving scores for classes

training

2-layer NN: $F = W_2 \max(0, W_1 x)$

3-layer NN: $F = W_3 \max(0, W_2 \max(0, W_1 x))$



10 numbers giving scores for classes

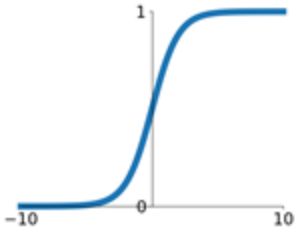
training

Activation function

Activation functions

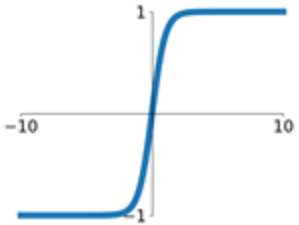
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



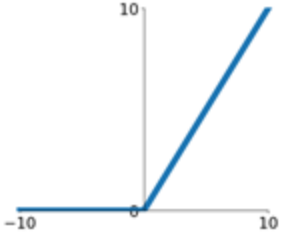
tanh

$$\tanh(x)$$



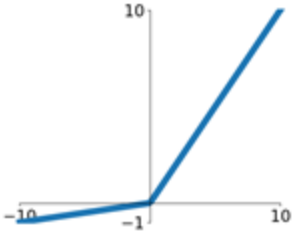
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

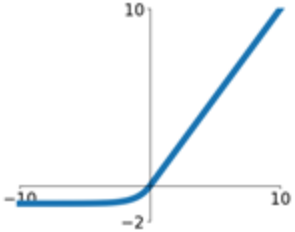


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



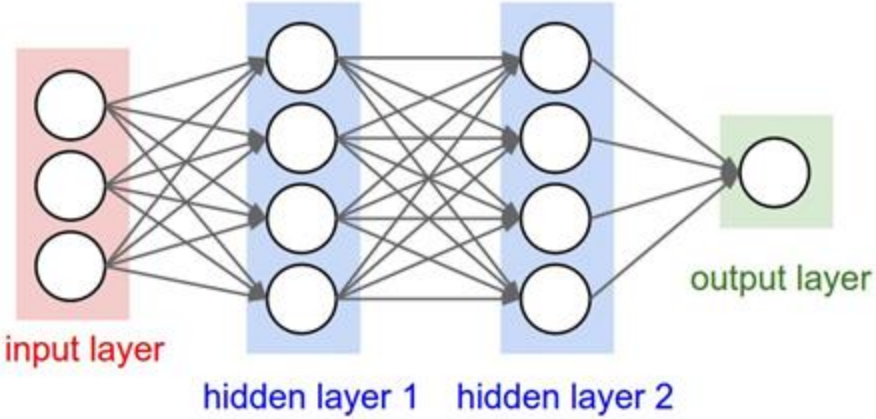
A. Alahi

Full implementation of training a 2-layer Neural Network needs ~20 lines:

```python
import numpy as np
from numpy.random import randn

N, D_in, H, D_out = 64, 1000, 100, 10
x, y = randn(N, D_in), randn(N, D_out)
w1, w2 = randn(D_in, H), randn(H, D_out)

for t in range(2000):
  h = 1 / (1 + np.exp(-x.dot(w1)))
  y_pred = h.dot(w2)
  loss = np.square(y_pred - y).sum()
  print(t, loss)

  grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h.T.dot(grad_y_pred)
  grad_h = grad_y_pred.dot(w2.T)
  grad_w1 = x.T.dot(grad_h * h * (1 - h))

  w1 -= 1e-4 * grad_w1
  w2 -= 1e-4 * grad_w2
```


A. Alahi

Example feed-forward computation of a neural network




```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

=> We can see NN as a function of function: F = $f o f o f o$ … where each f is non-linear

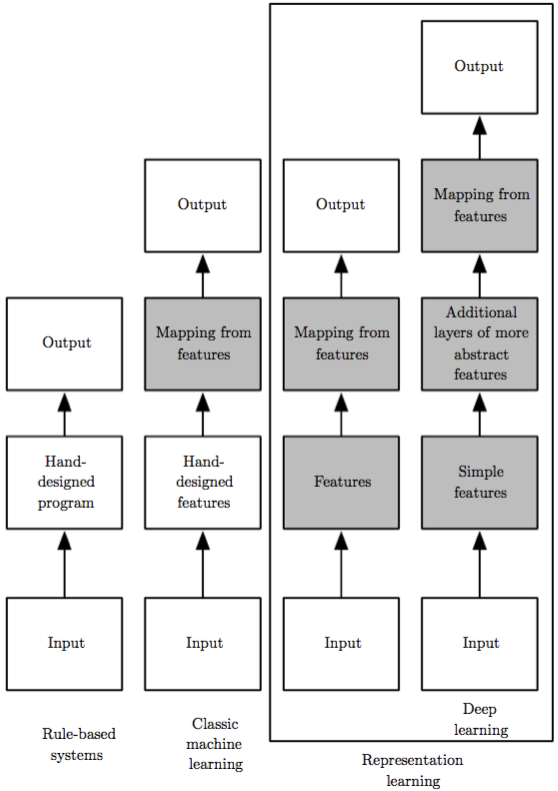
A. Alahi

So Far…

We saw:

- How to learn the mapping from an input (a representation) to an output (with optimization methods…)
- How to also discover the representation itself **with Neural Network** (referred to **representation learning)**

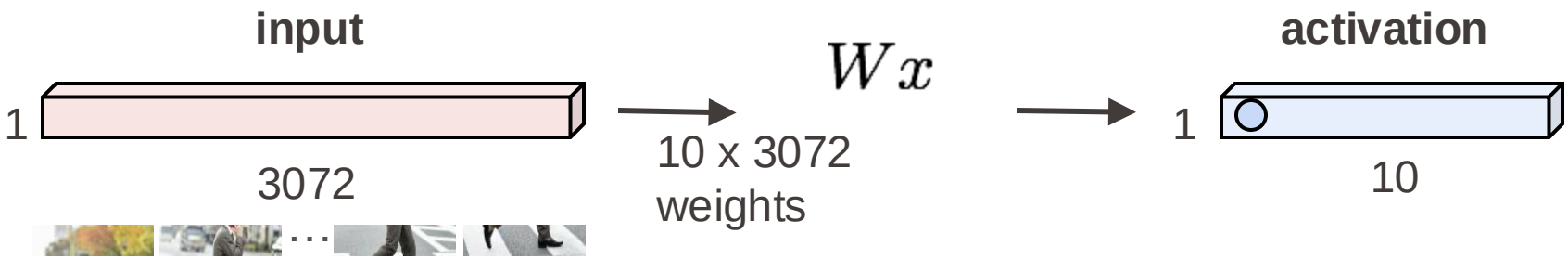
- **Now,** we will see a family of NN used for perception tasks: **Convolutional Neural Networks**



Convolutional Neural Networks (CNN)

Limitation of a linear function

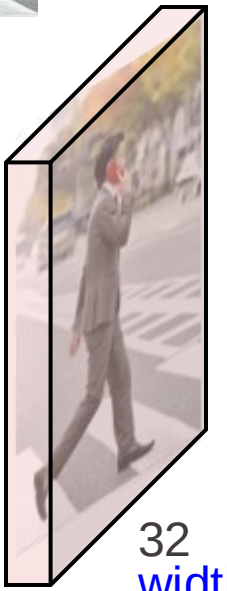
32x32x3 image -> stretch to 3072 x 1



How to efficiently model correlation between neighboring pixels?

Convolution Layer

32x32x3 image -> preserve spatial structure



32
height

32
width

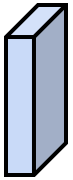
3
depth

A. Alahi

Convolution Layer

32x32x3 image

5x5x3 filter

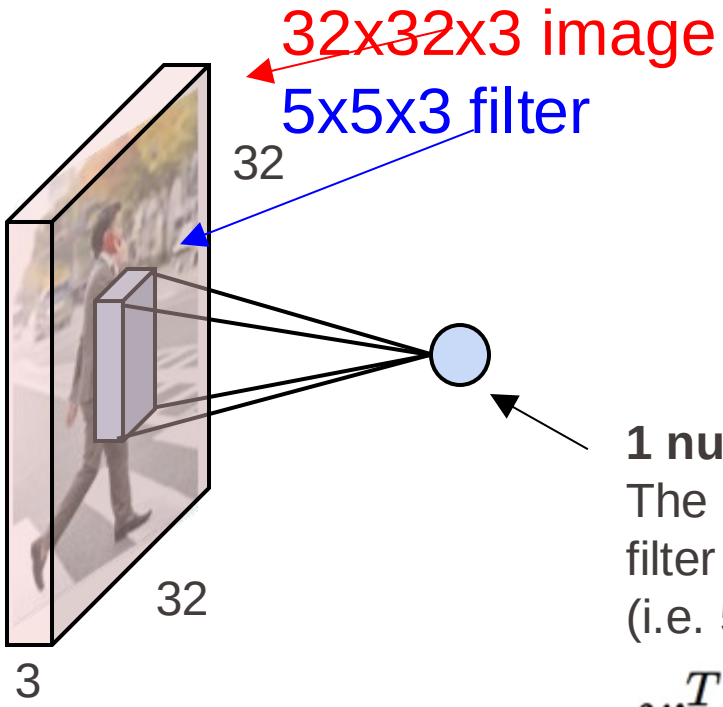


Convolve the filter with the image i.e., “slide over the image spatially, computing dot products”

A. Alahi

Convolution Layer

A. Alahi



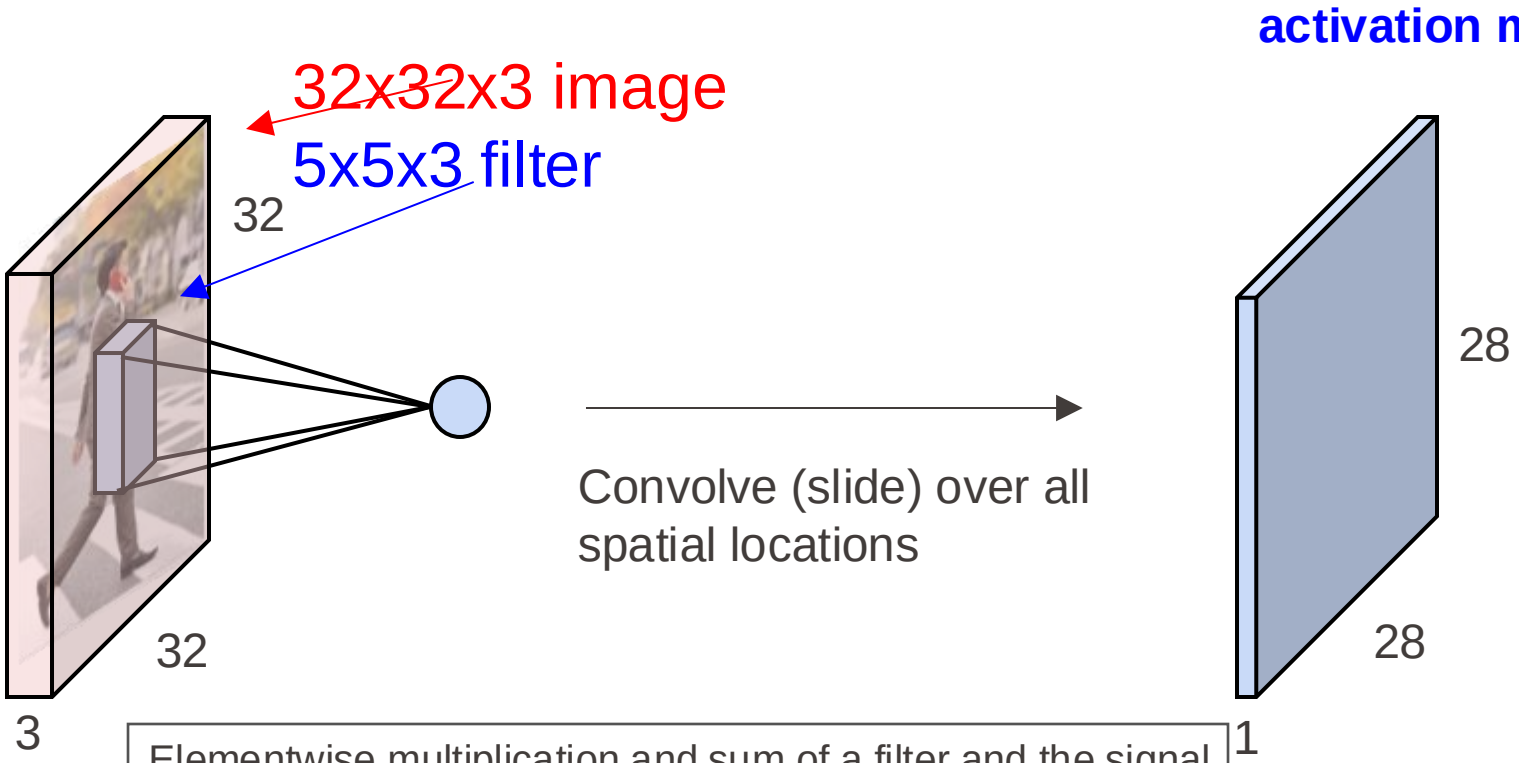
w

1 number:
The result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

Convolution Layer

A. Alahi

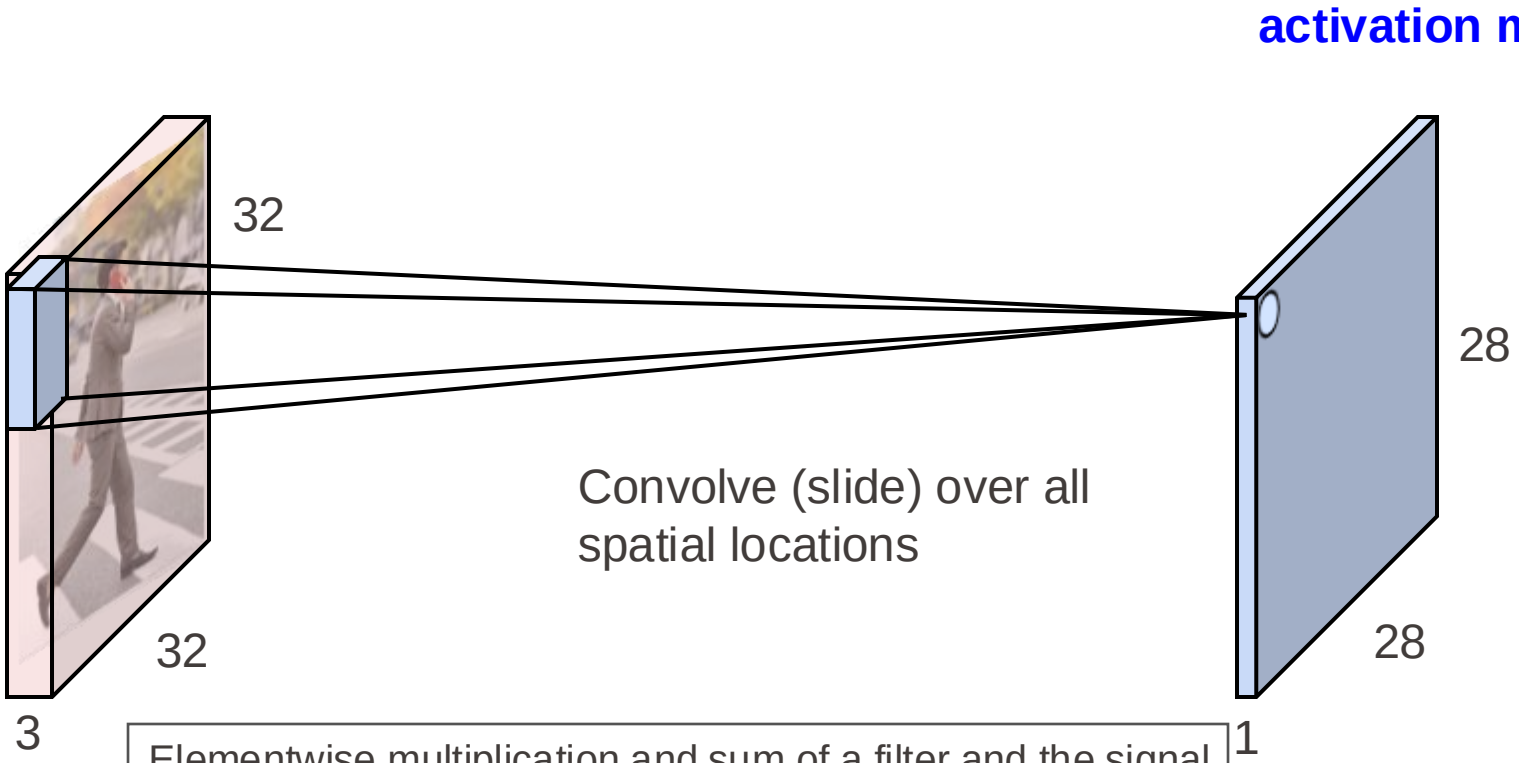


Elementwise multiplication and sum of a filter and the signal (image)

$$f[x,y]*w[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2]\cdot w[x-n_1,y-n_2]$$

Convolution Layer

A. Alahi



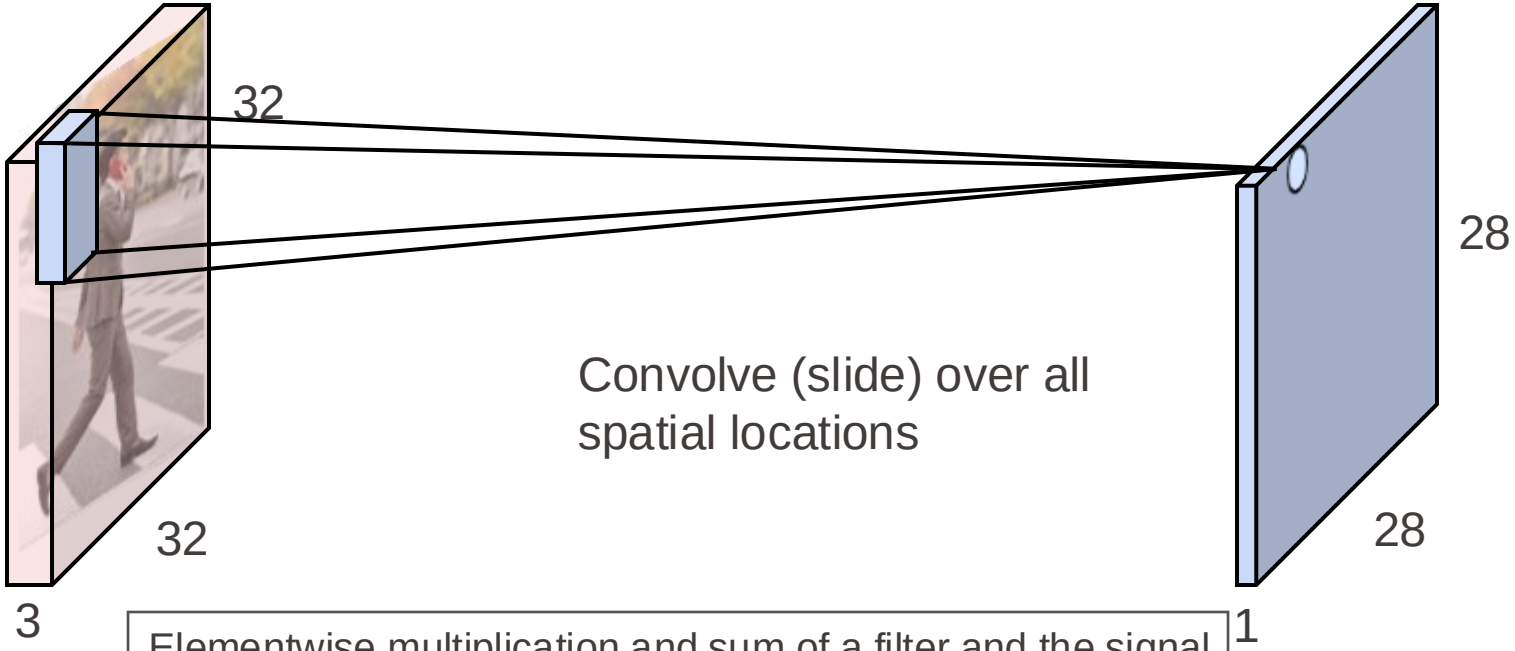
Elementwise multiplication and sum of a filter and the signal (image)

$$f[x,y]*w[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2]\cdot w[x-n_1,y-n_2]$$

Convolution Layer

A. Alahi

activation map



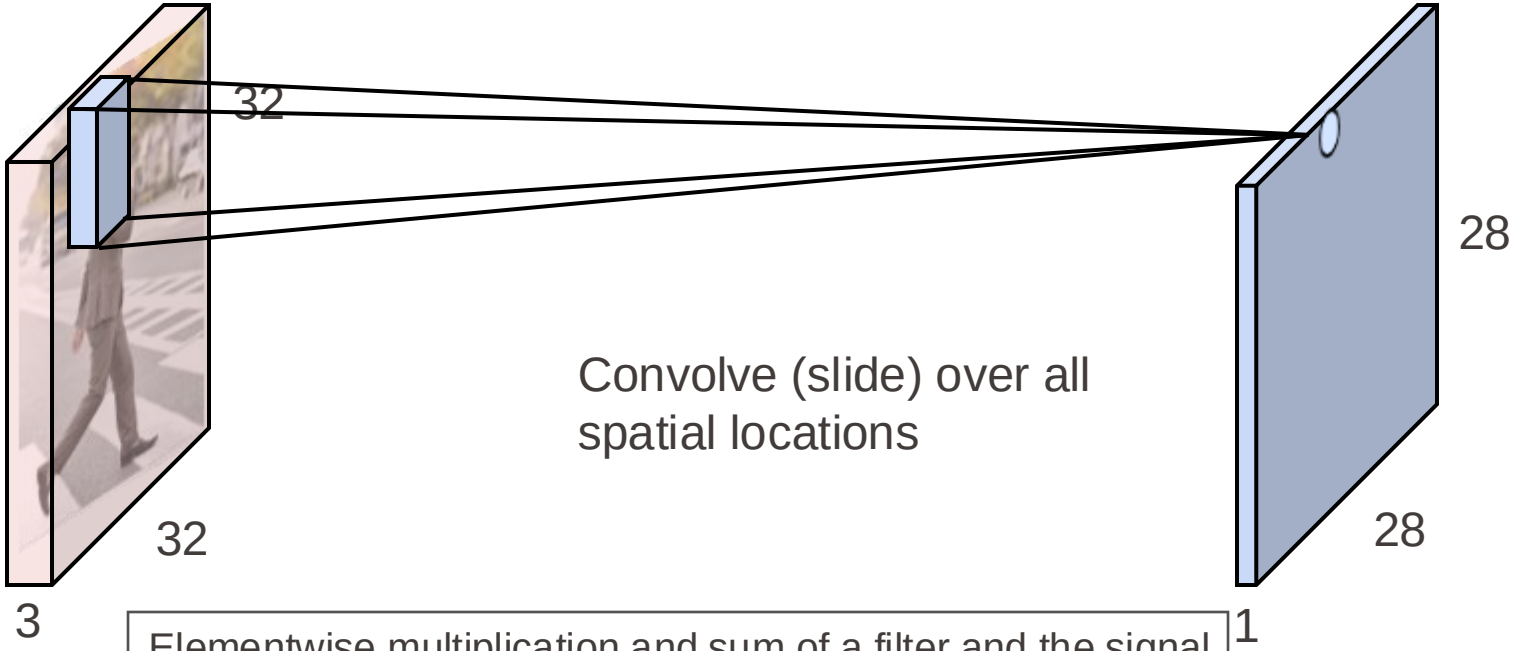
Elementwise multiplication and sum of a filter and the signal (image)

$$f[x,y]*w[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2]\cdot w[x-n_1,y-n_2]$$

Convolution Layer

A. Alahi

activation map



Convolve (slide) over all spatial locations

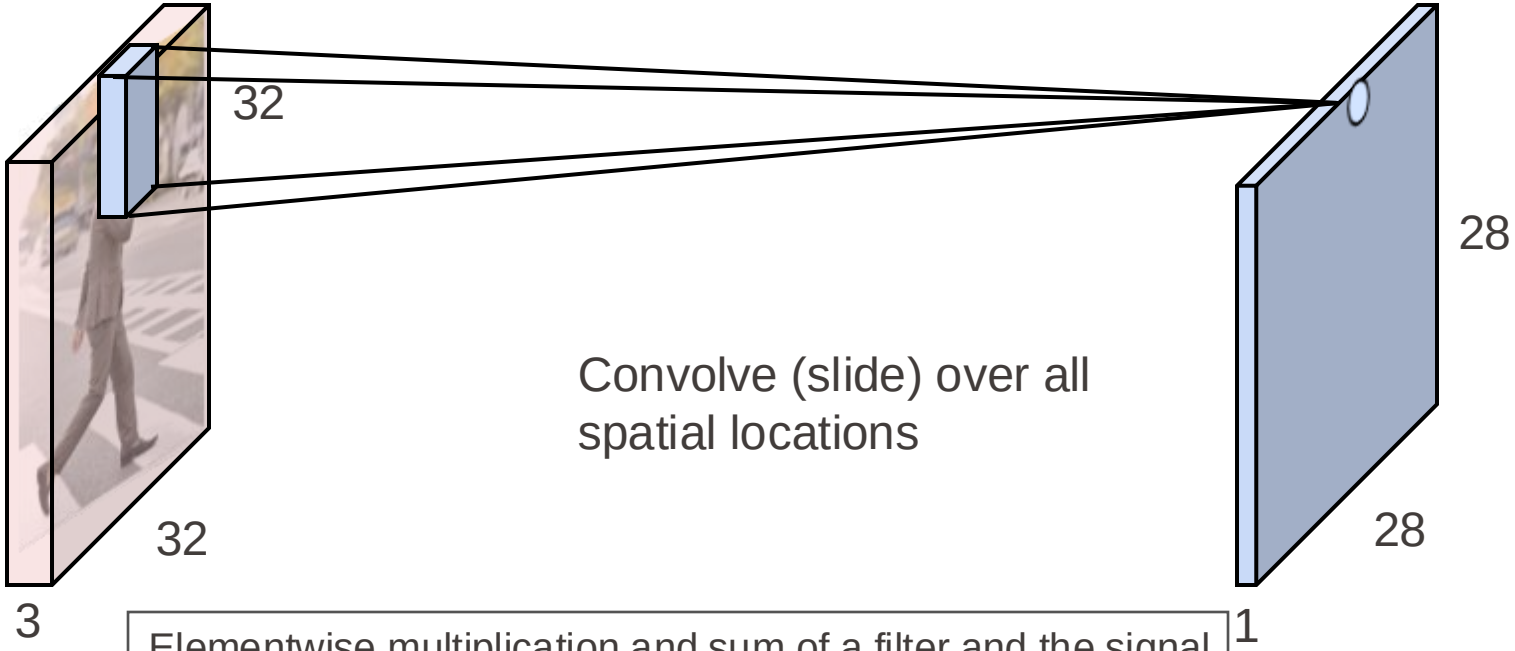
Elementwise multiplication and sum of a filter and the signal (image)

$$f[x,y]*w[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2]\cdot w[x-n_1,y-n_2]$$

Convolution Layer

A. Alahi

activation map



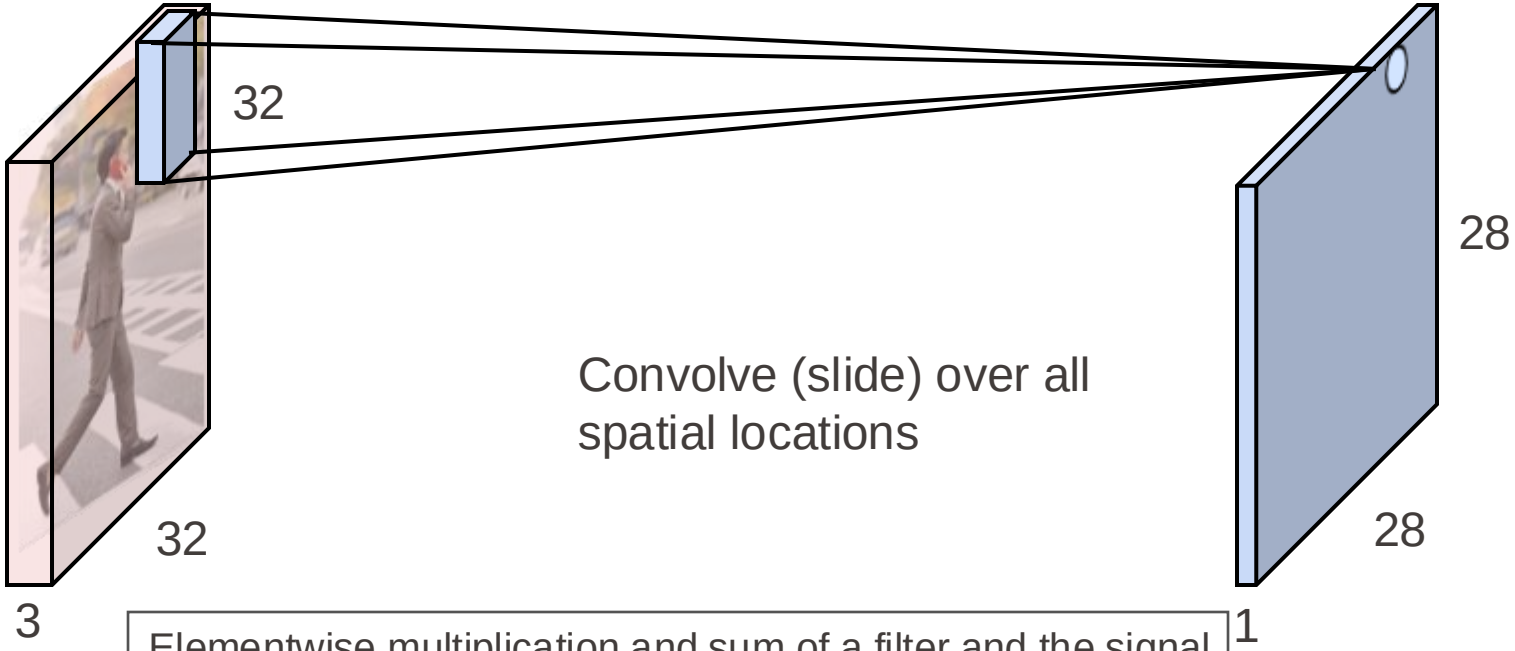
Convolve (slide) over all spatial locations

Elementwise multiplication and sum of a filter and the signal (image)

$$f[x,y]*w[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2]\cdot w[x-n_1,y-n_2]$$

Convolution Layer

activation map



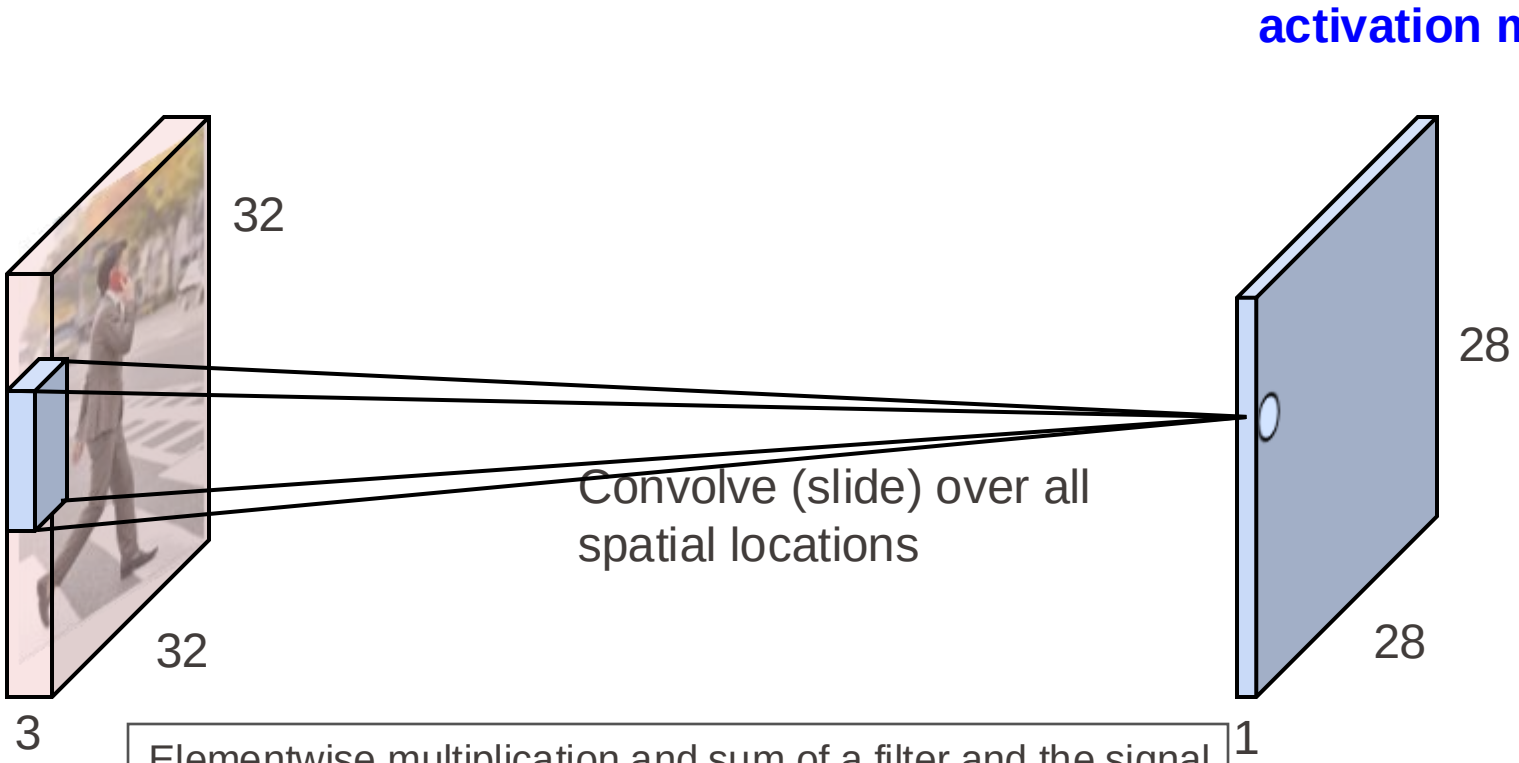
Elementwise multiplication and sum of a filter and the signal (image)

$$f[x,y]*w[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2]\cdot w[x-n_1,y-n_2]$$

A. Alahi

Convolution Layer

A. Alahi



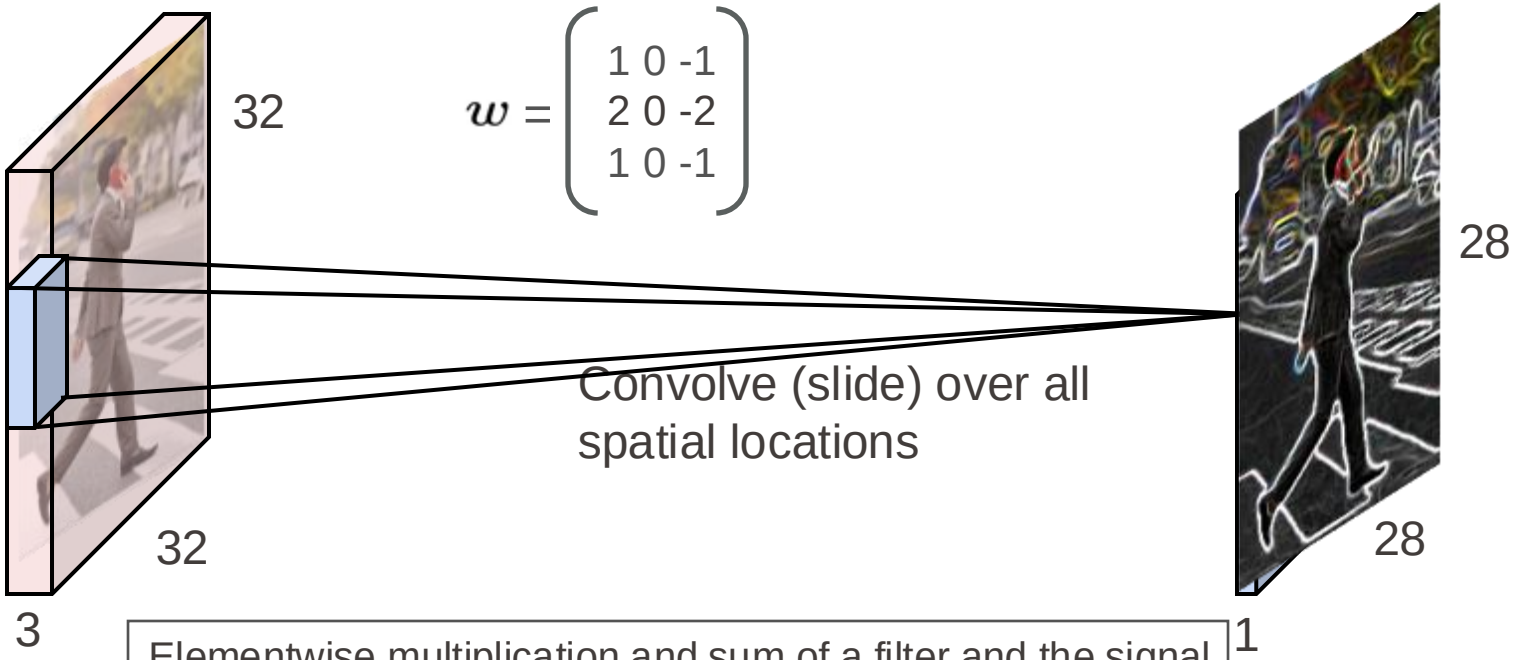
Elementwise multiplication and sum of a filter and the signal (image)

$$f[x,y]*w[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2]\cdot w[x-n_1,y-n_2]$$

Convolution Layer

A. Alahi

activation map



$$w = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{pmatrix}$$

Elementwise multiplication and sum of a filter and the signal (image)

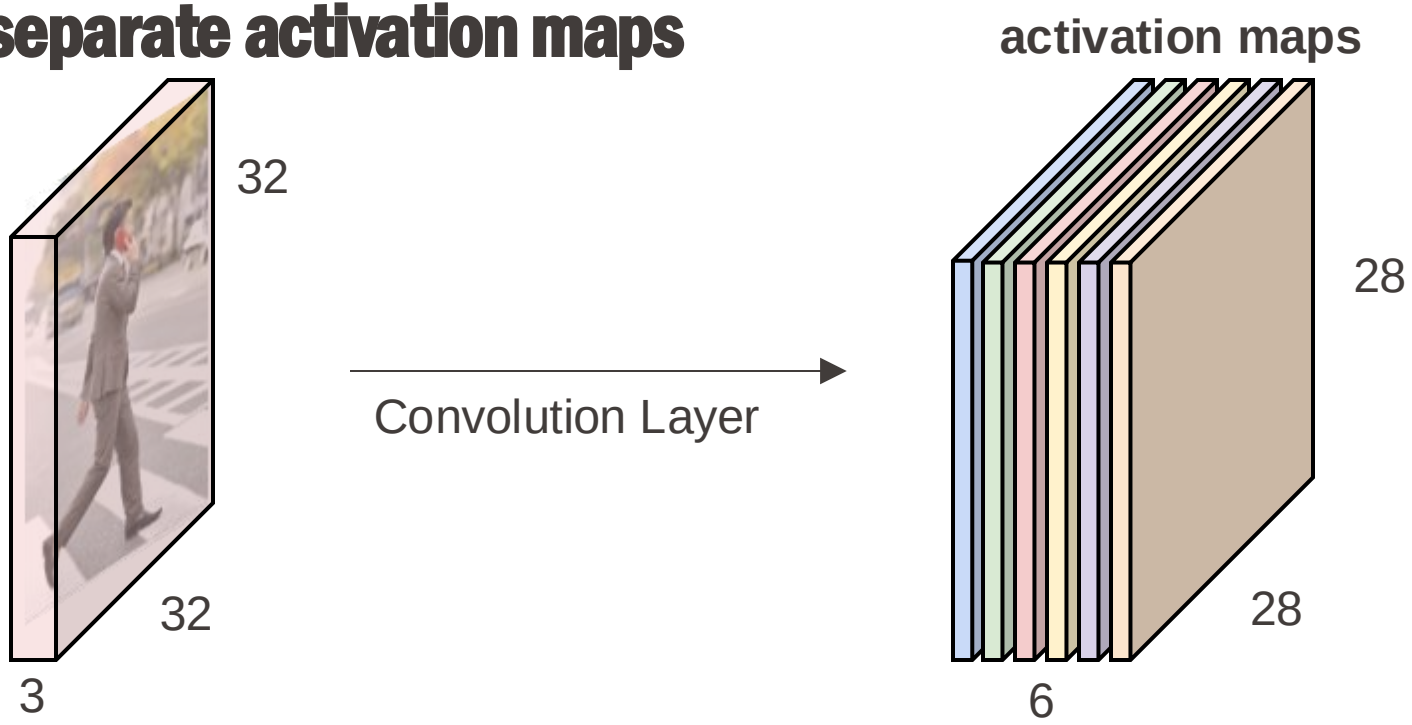
$$f[x,y] * w[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2] \cdot w[x-n_1, y-n_2]$$

Convolution Layer

A. Alahi

Consider a second, green filter

With 6 5x5 filters, we'll get 6 separate activation maps

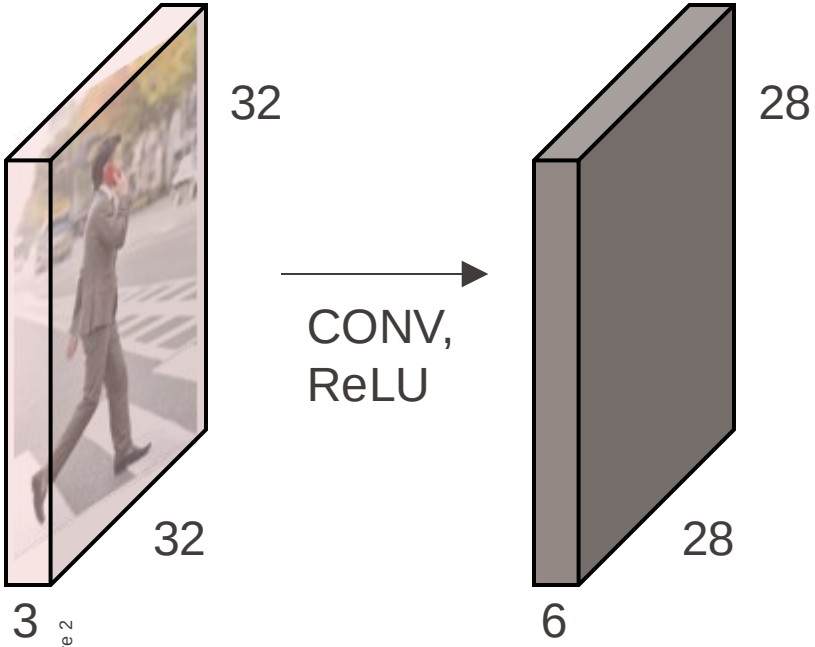


We stack these up to get a "new image" of size 28x28x6!

tensor

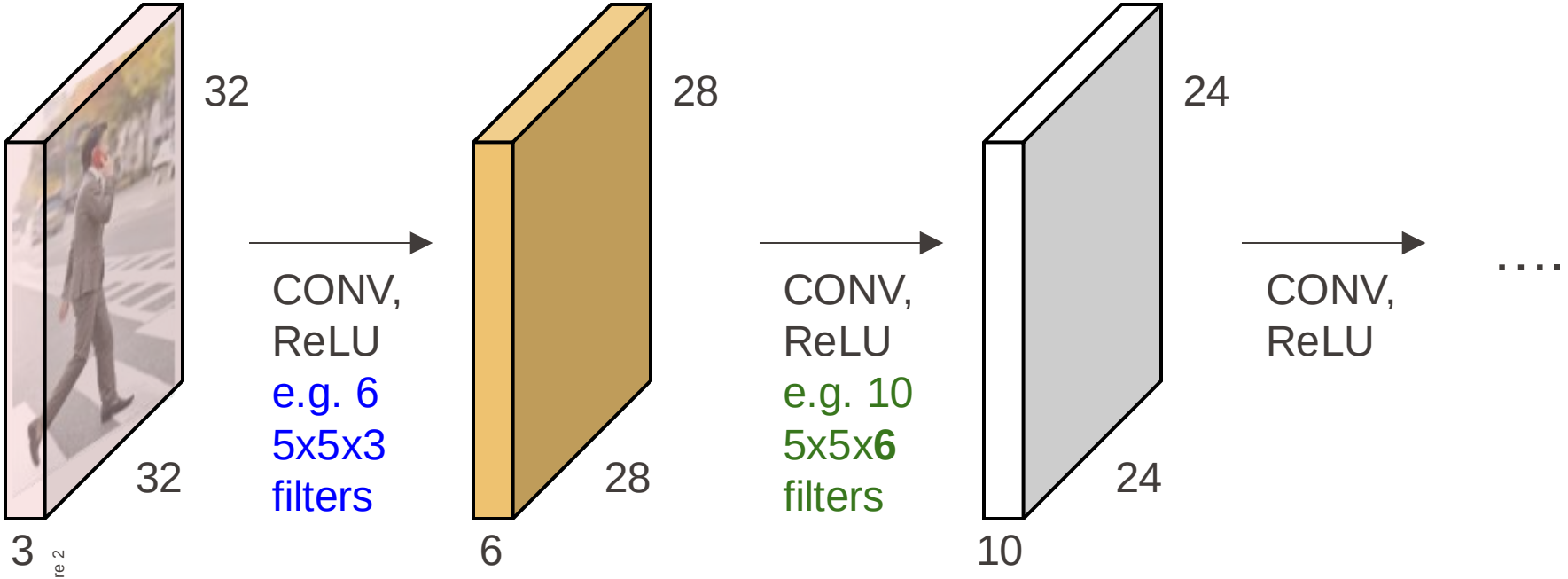
A. Alahi

CNN is a sequence of Convolutional Layers, interspersed with activation functions



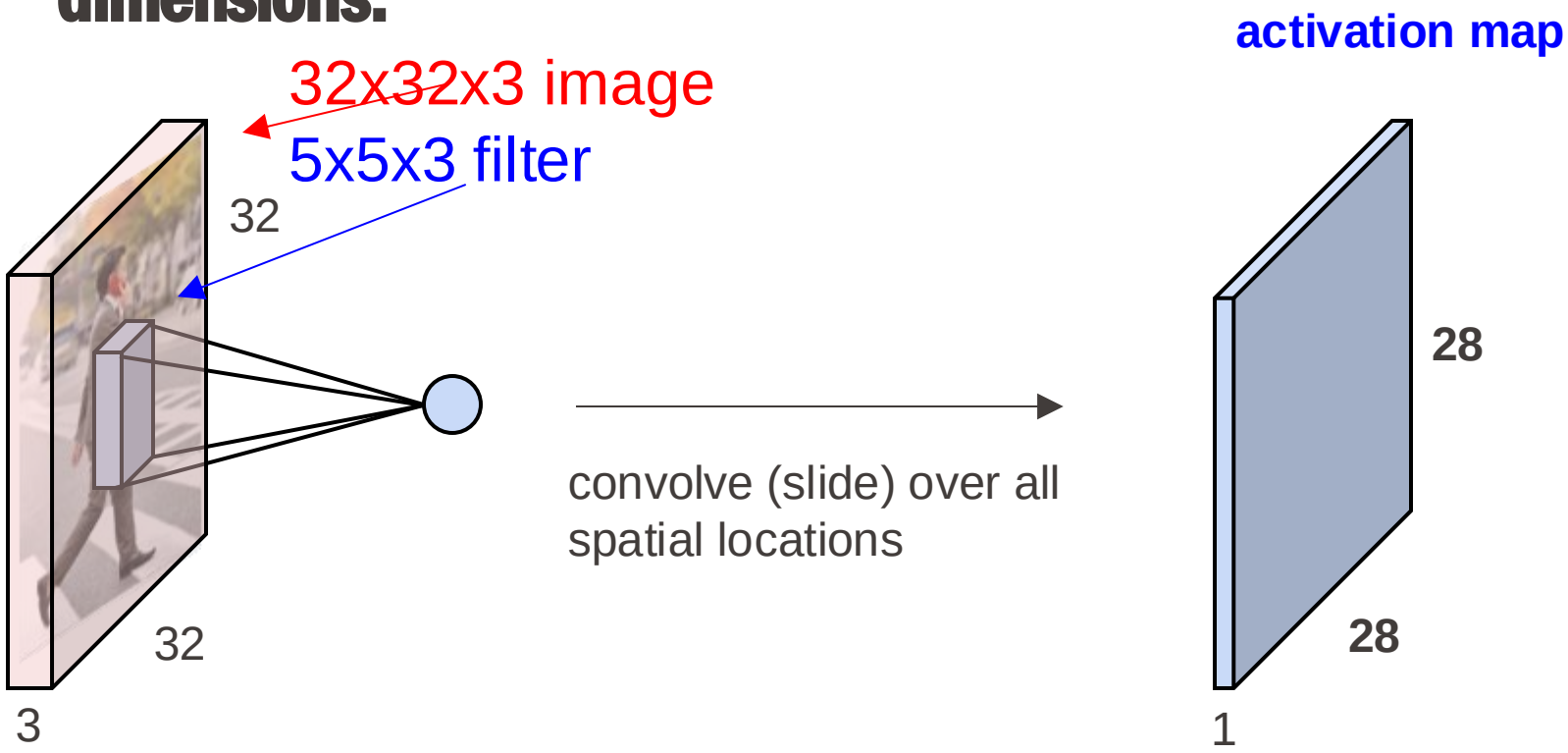
A. Alahi

CNN is a sequence of Convolutional Layers, interspersed with activation functions



A. Alahi

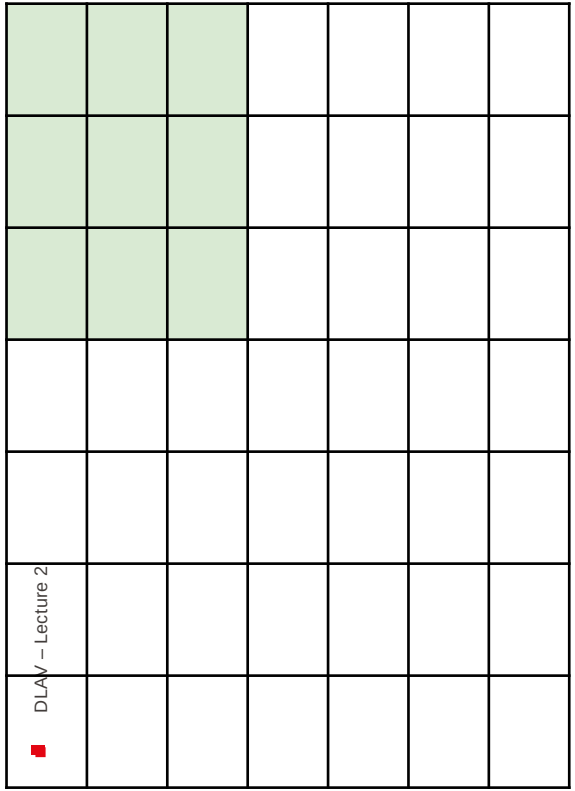
A closer look at spatial dimensions:



A. Alahi

A closer look at spatial dimensions:

7



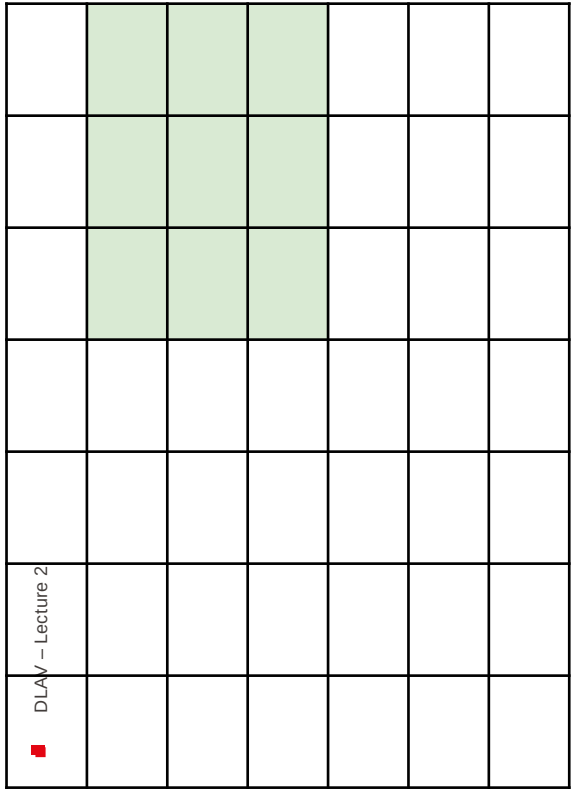
7

7x7 input (spatially)
assume 3x3 filter

A. Alahi

A closer look at spatial dimensions:

7



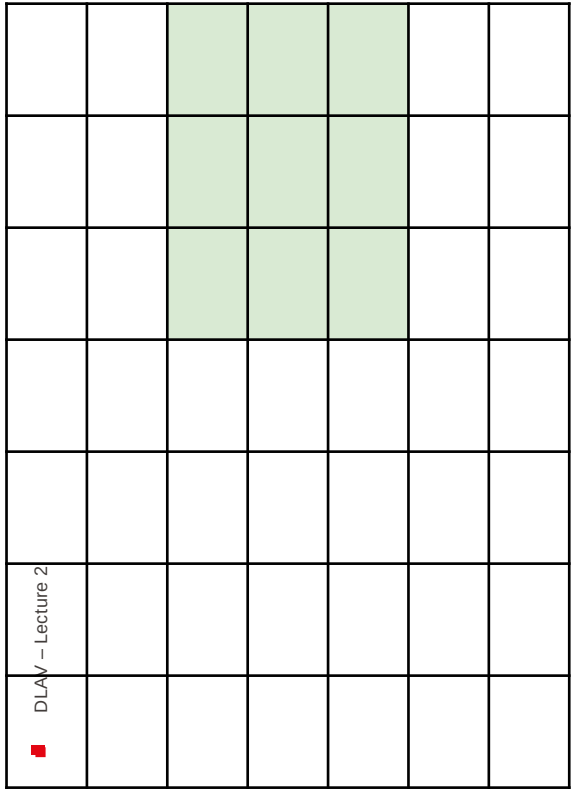
7

7x7 input (spatially)
assume 3x3 filter

A. Alahi

A closer look at spatial dimensions:

7



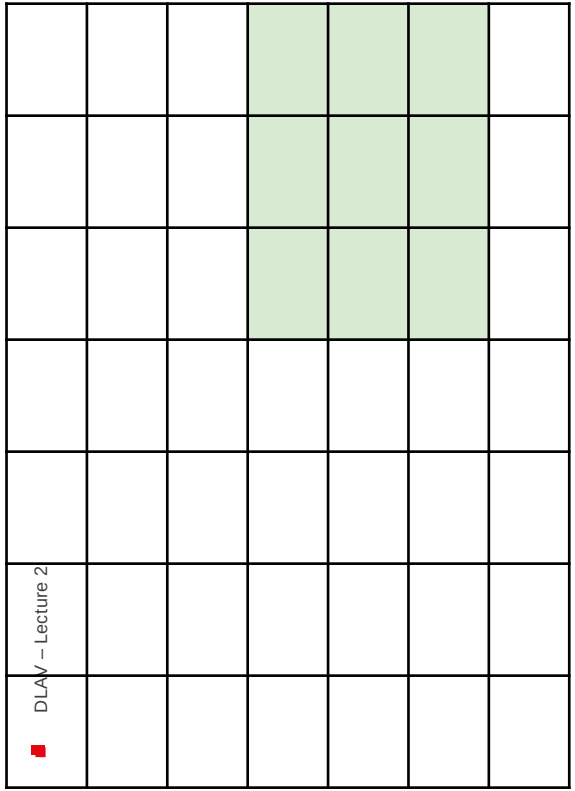
7

7x7 input (spatially)
assume 3x3 filter

A. Alahi

A closer look at spatial dimensions:

7



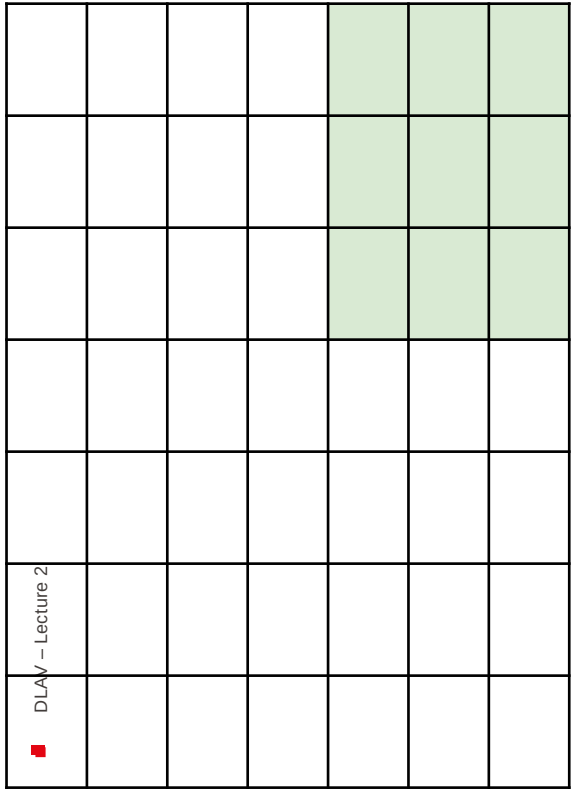
7

7x7 input (spatially)
assume 3x3 filter

A. Alahi

A closer look at spatial dimensions:

7



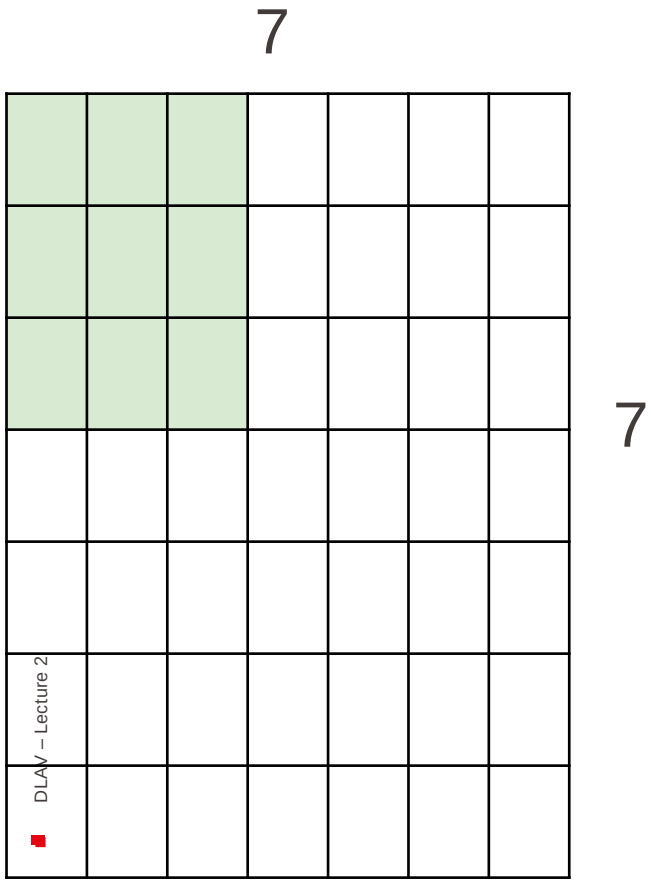
7

7x7 input (spatially)
assume 3x3 filter

=> 5x5 output

A. Alahi

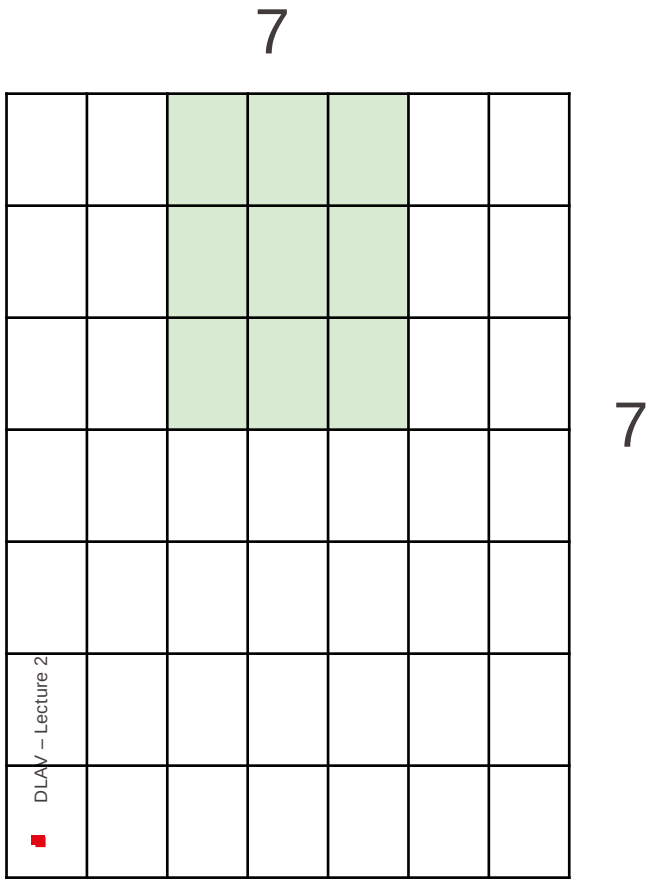
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A. Alahi

A closer look at spatial dimensions:

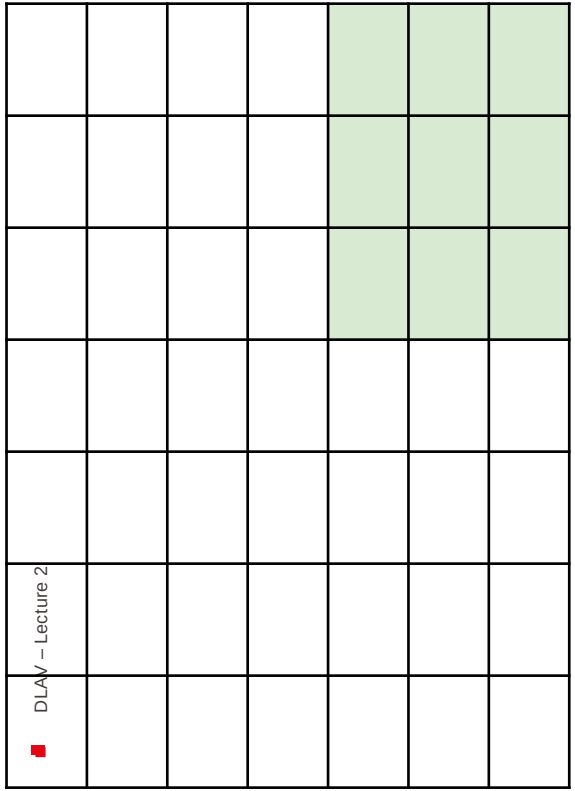


7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A. Alahi

A closer look at spatial dimensions:

7



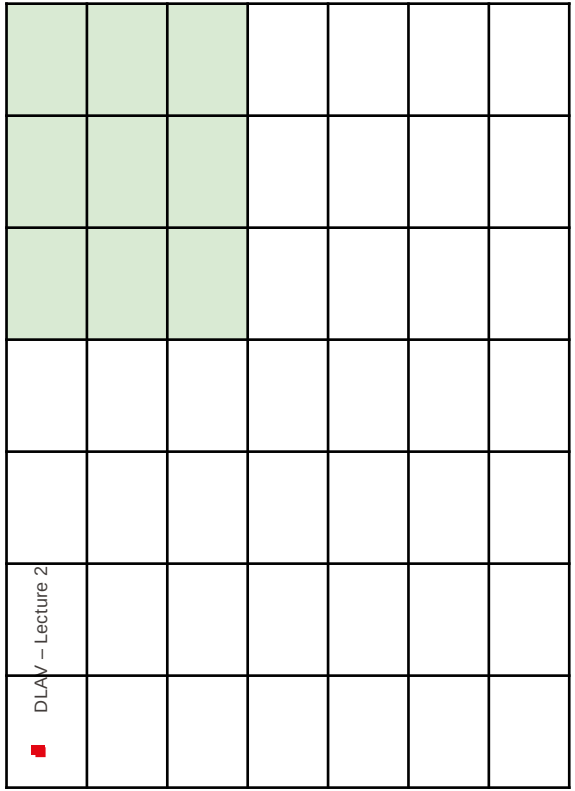
7

7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

A. Alahi

A closer look at spatial dimensions:

7



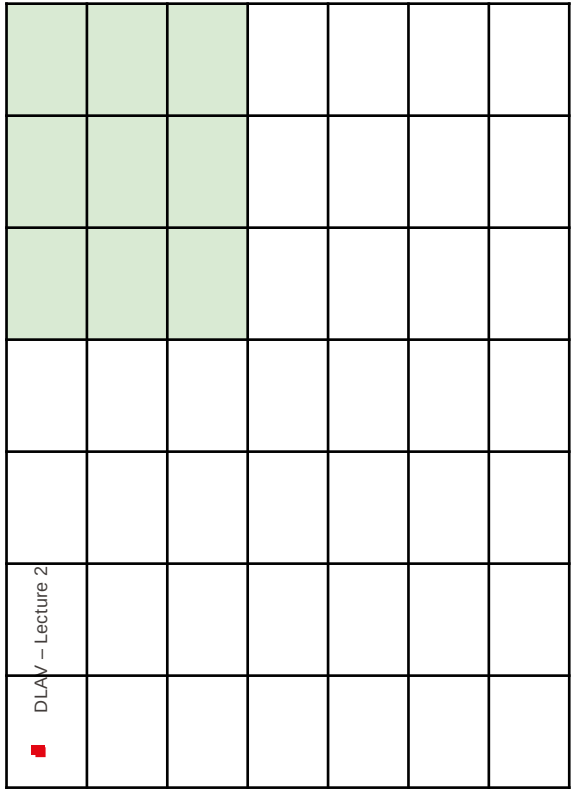
7

7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

A. Alahi

A closer look at spatial dimensions:

7

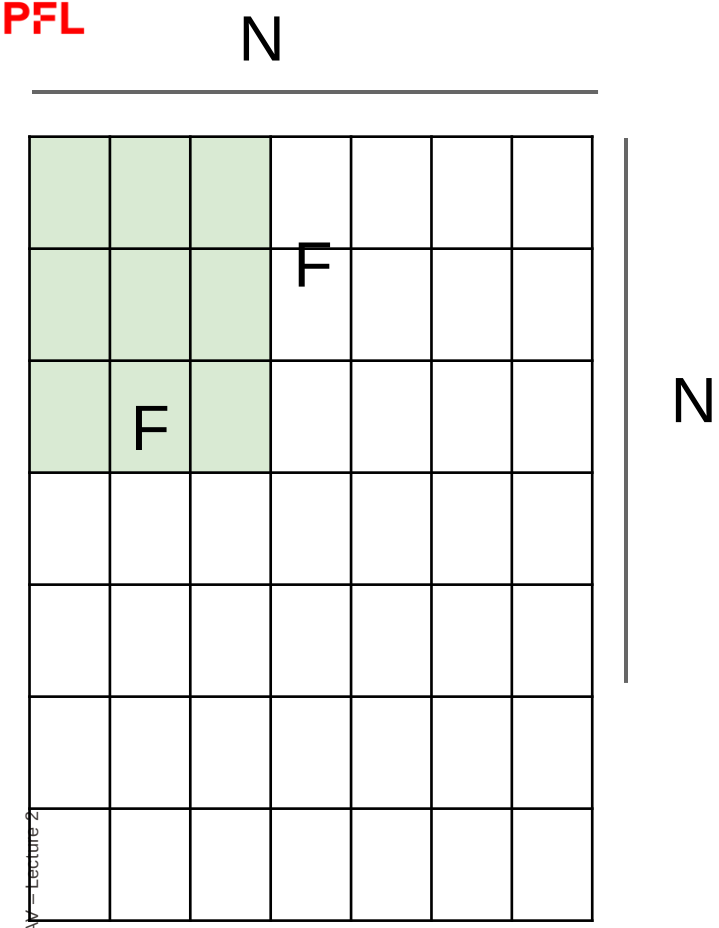


7

7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.

A. Alahi



Output size:

(N - F) / stride + 1

e.g., N = 7, F = 3:

stride 1 => (7 - 3)/1 + 1 = 5

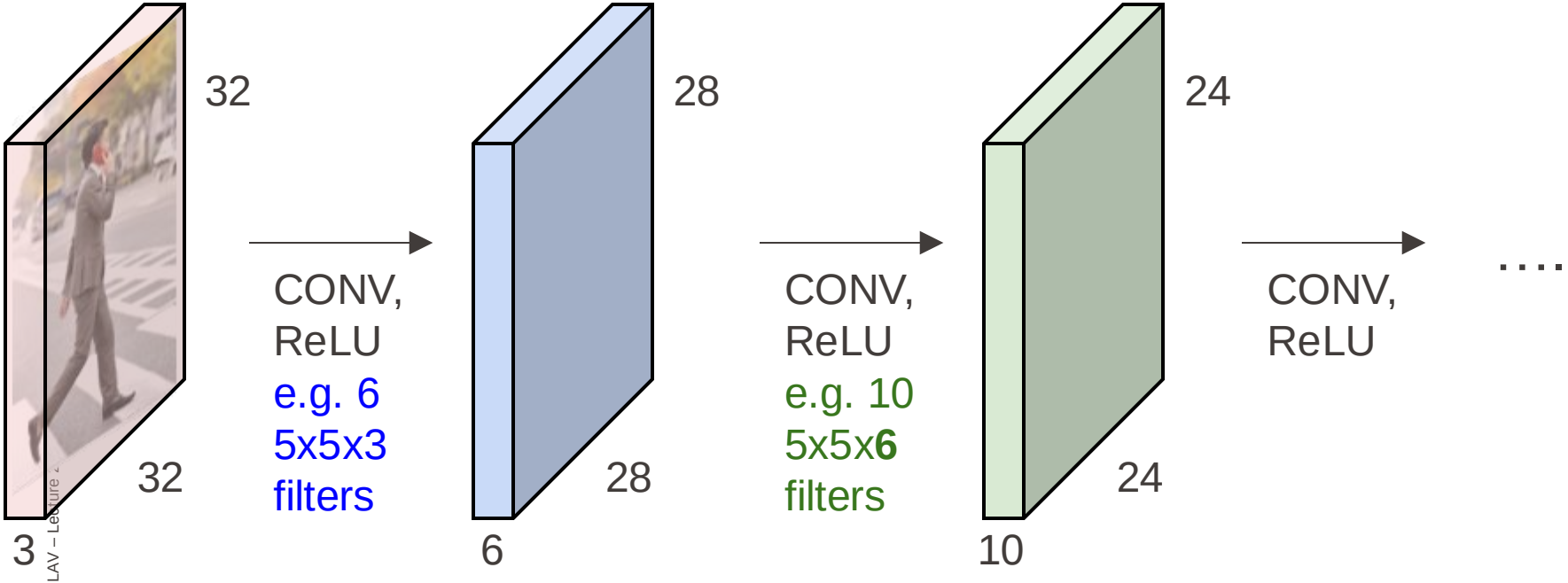
stride 2 => (7 - 3)/2 + 1 = 3

stride 3 => (7 - 3)/3 + 1 = 2.33 :\

A. Alahi

Back to our example…

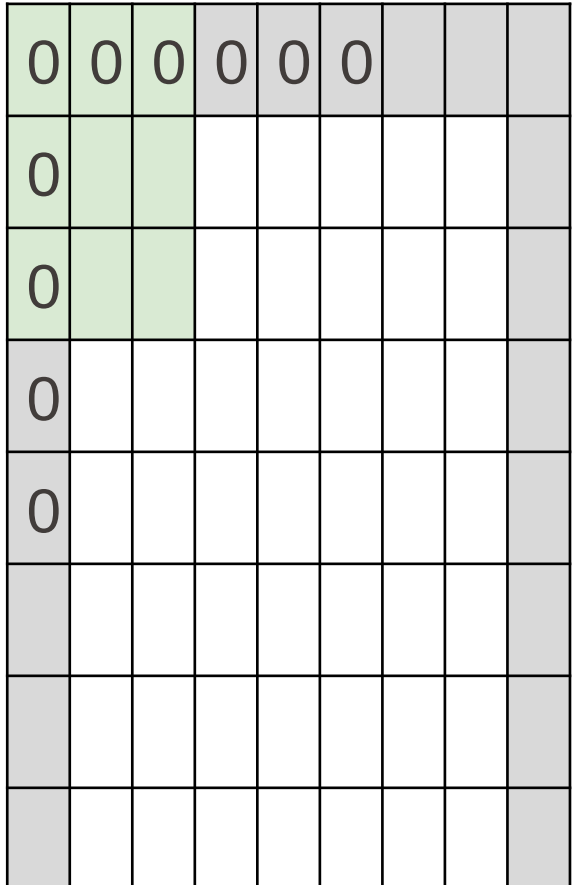
A 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially!
(32 -> 28 -> 24 ...). Shrinking too fast is not good, doesn't work well.



A. Alahi

In practice: Common to zero pad the border

A. Alahi

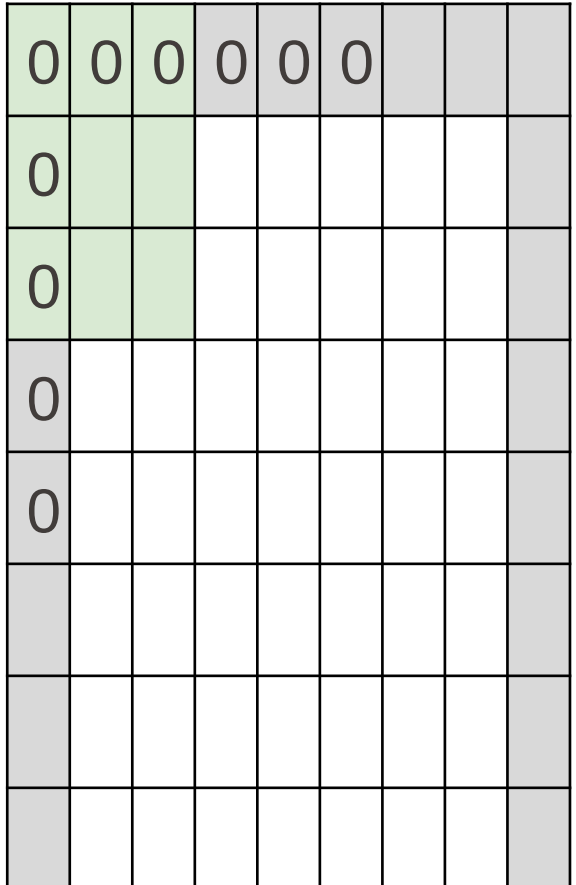


e.g., input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

(recall:)
(N - F) / stride + 1

In practice: Common to zero pad the border

A. Alahi

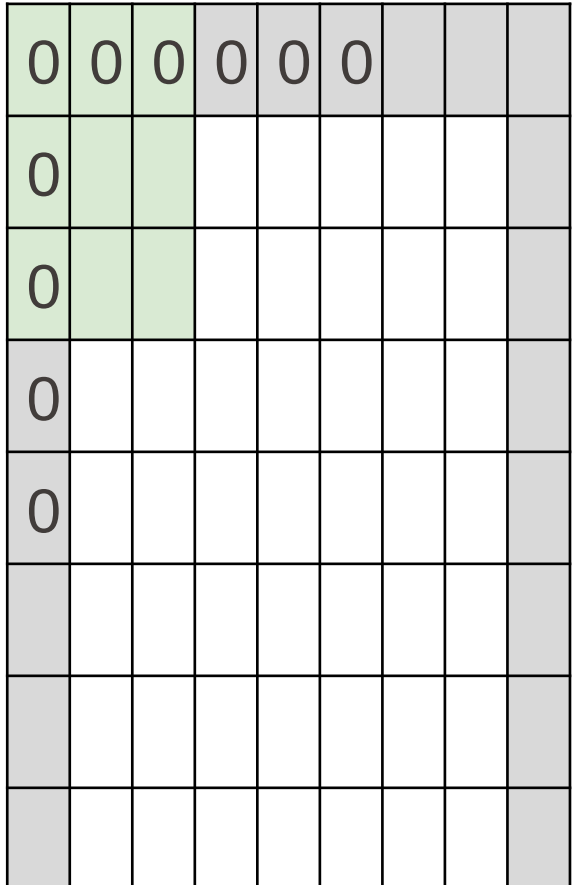


e.g., input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!

In practice: Common to zero pad the border

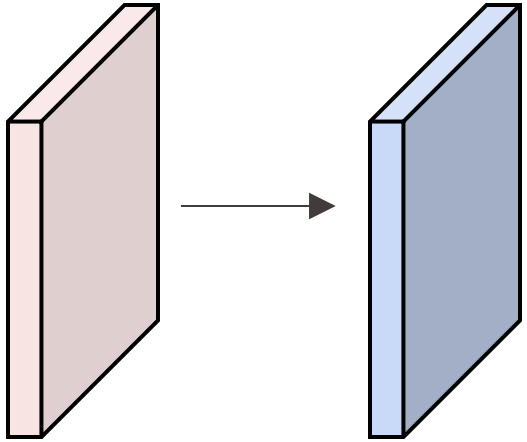
A. Alahi



e.g., input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

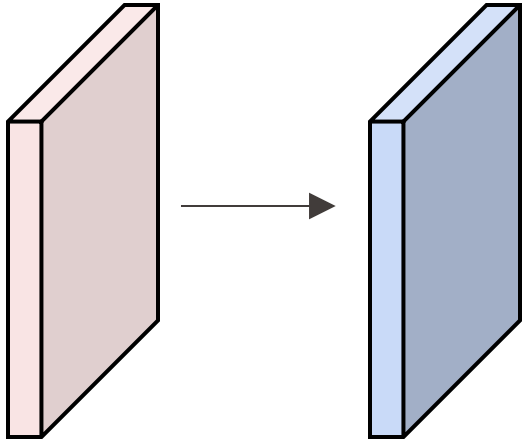
With padding



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size: ?

With padding



Input volume: **32x32x3**

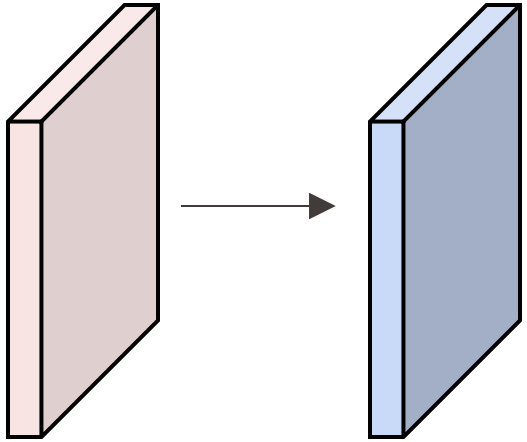
10 5x5 filters with stride 1, pad 2

Output volume size:

(32+2*2-5)/1+1 = 32 spatially, so

32x32x10

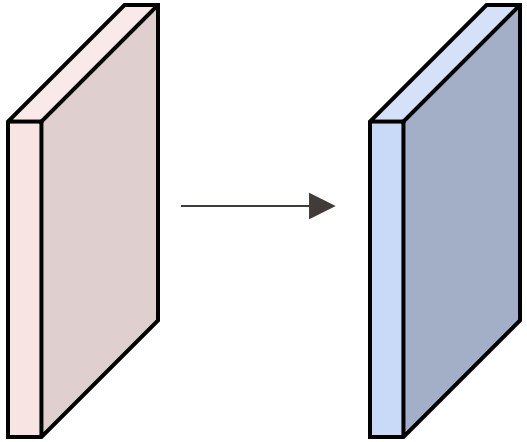
With padding



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?

With padding



Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?

each filter has 5*5*3 + 1 = 76 params (+1 for bias)

=> 76*10 = **760**

A. Alahi

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d-th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d-th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

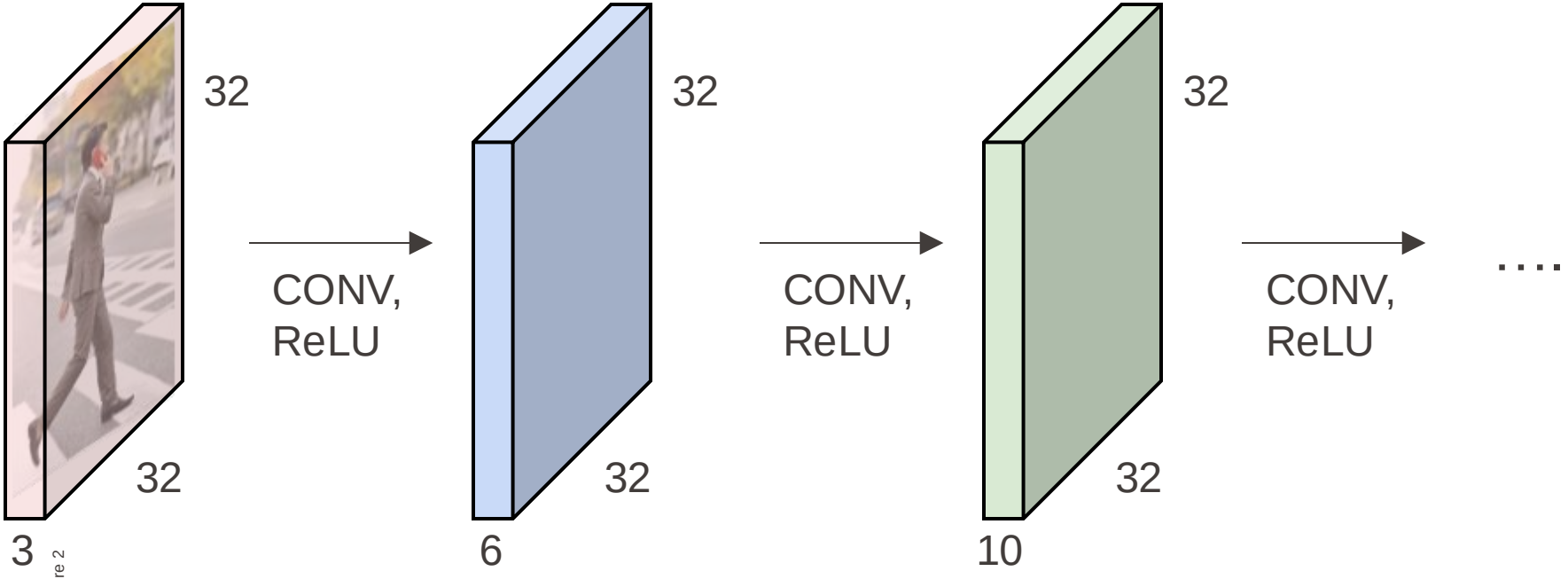
Common settings:

K = (powers of 2, e.g. 32, 64, 128, 512)

- F = 3, S = 1, P = 1
- F = 5, S = 1, P = 2
- F = 5, S = 2, P = ? (whatever fits)
- F = 1, S = 1, P = 0

A. Alahi

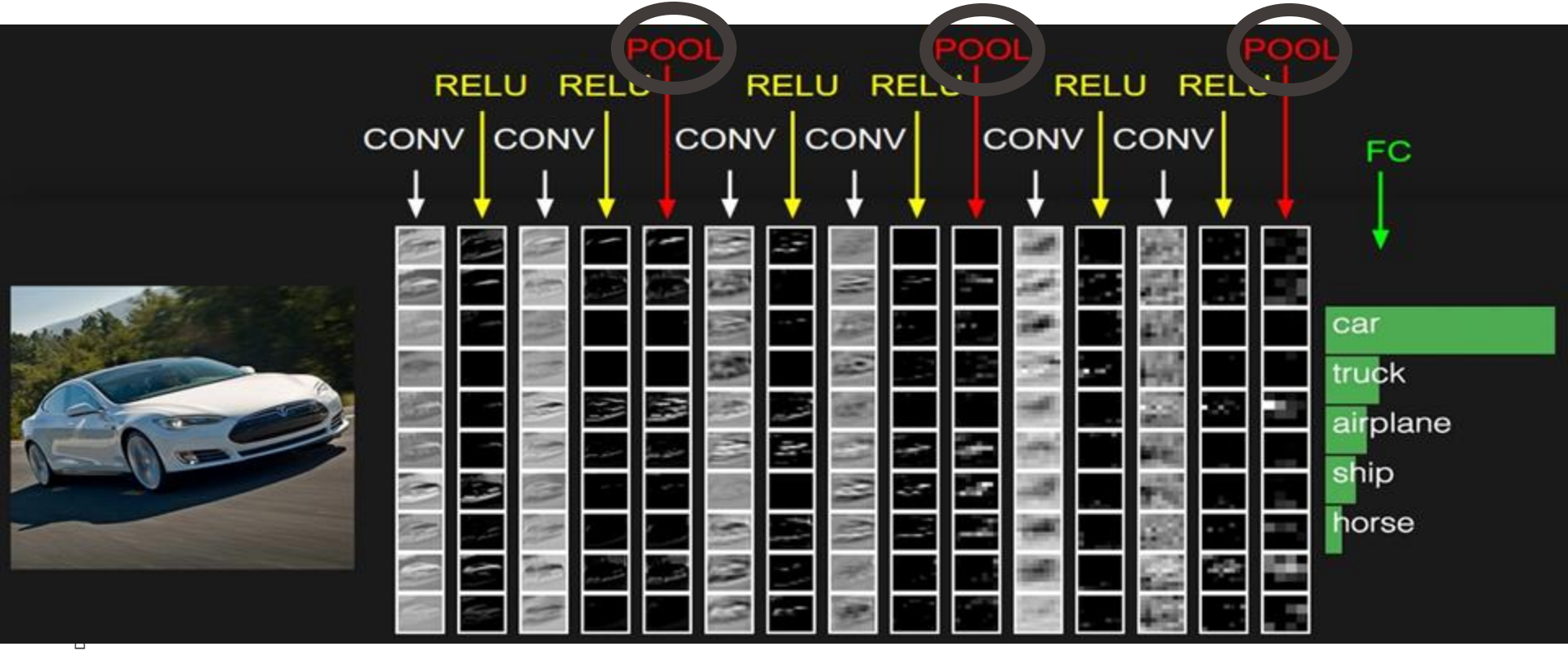
SO FAR…CNN is a sequence of Convolutional Layers, interspersed with activation functions



More layers can be used!

A. Alahi

Example of a CNN architecture



A. Alahi

MAX POOLING

A. Alahi

Single depth slice

x

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

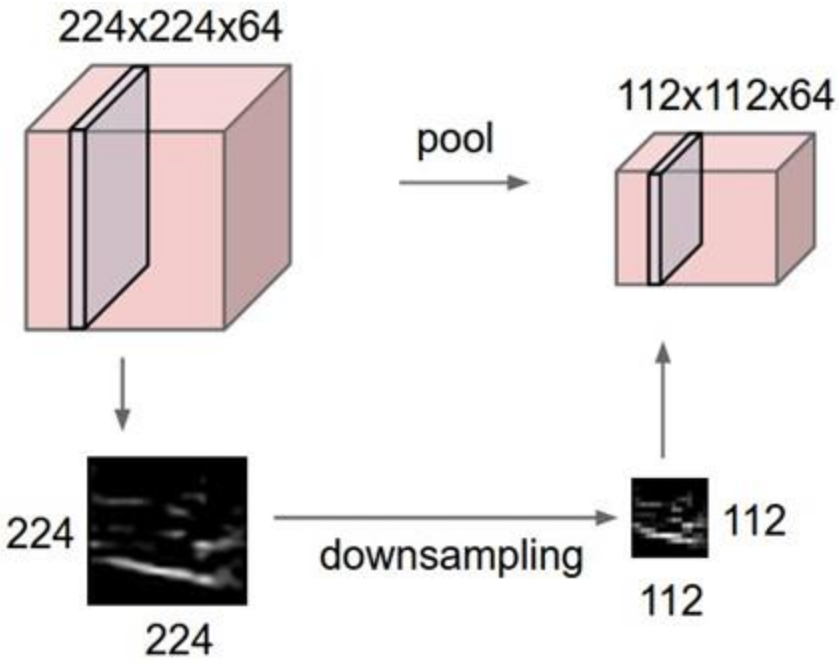
y

max pool with 2x2 filters
and stride 2

6	8
3	4

Pooling layer

Makes the representations smaller and more manageable
Operates over each activation map independently:



A. Alahi

MAX POOLING

A. Alahi

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for Pooling layers

MAX POOLING

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for Pooling layers

Common settings:

F = 2, S = 2
F = 3, S = 2

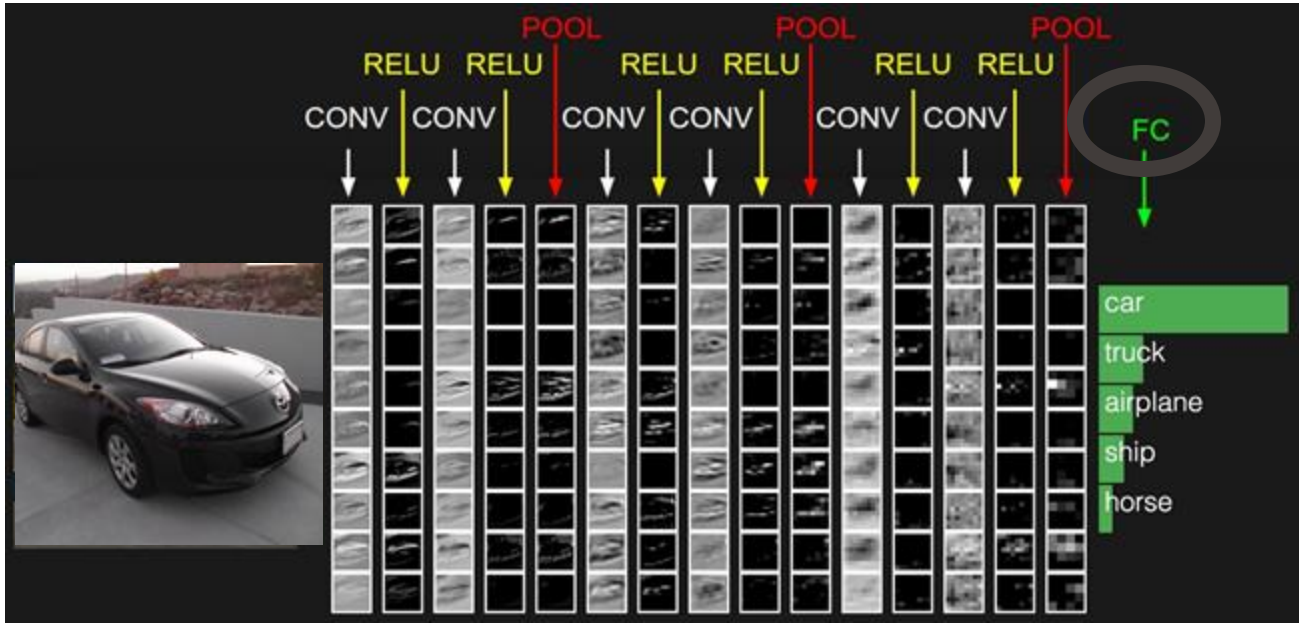
A. Alahi

Example of a CNN architecture

A. Alahi

Fully Connected Layer (FC layer)

- Contains neurons that connect to the entire input volume, as in ordinary Neural Networks



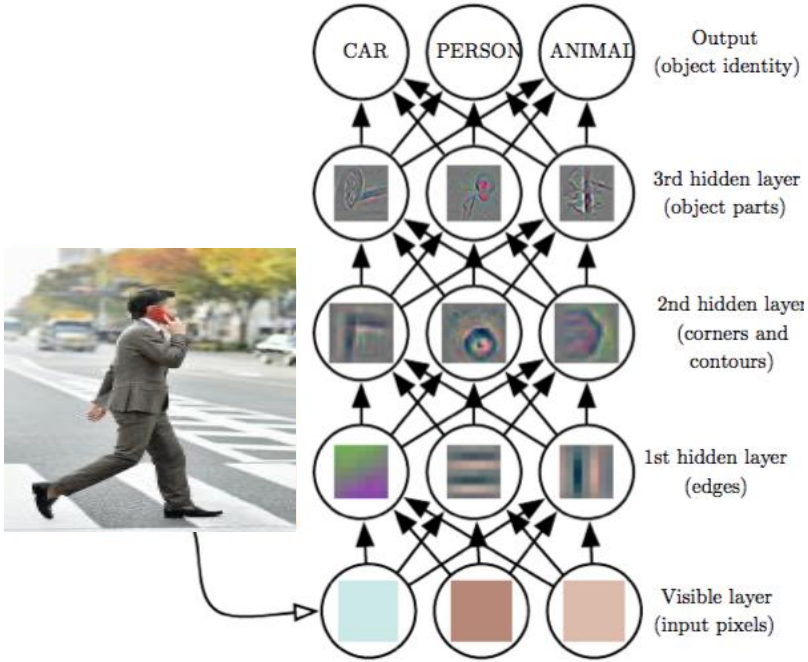
So far…

- Convolutional Neural Networks have multiple layers made of:
  - Convolution layer
  - Pooling layer
  - Fully connected layer

- Yet, the challenge is to design the architecture:
  - How many layers?
  - Filter size? How many?
  - Etc…

Intuitions behind Convolutional Neural Networks (CNN)

CNN
- Intuitions -



High level features

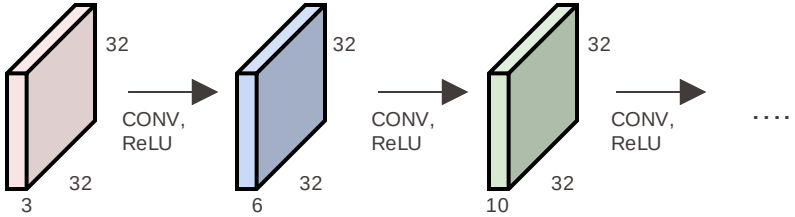
↑

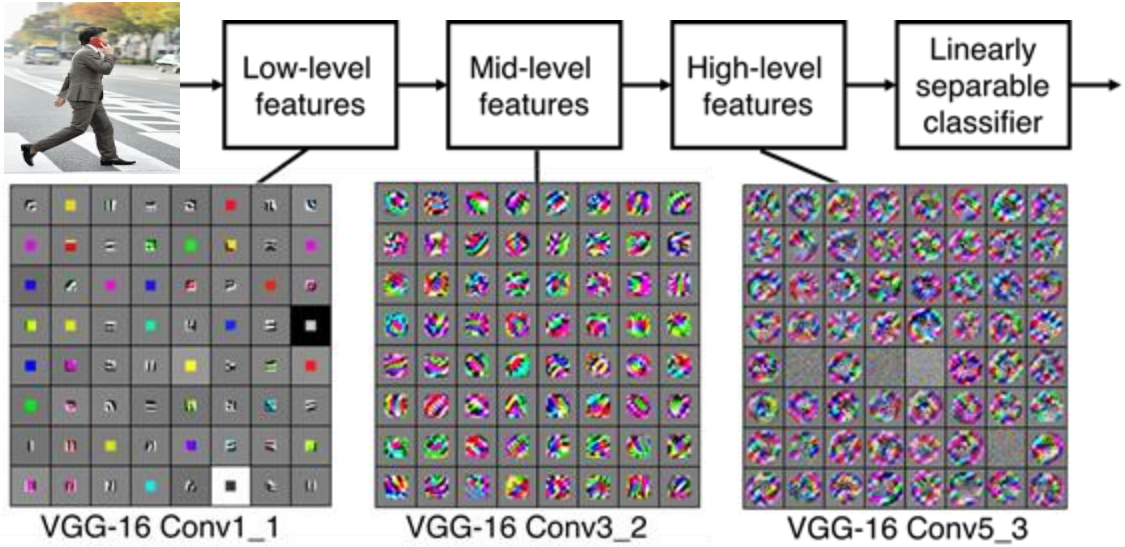
Mid level features

↑

Low level features

CNN
- Intuitions -

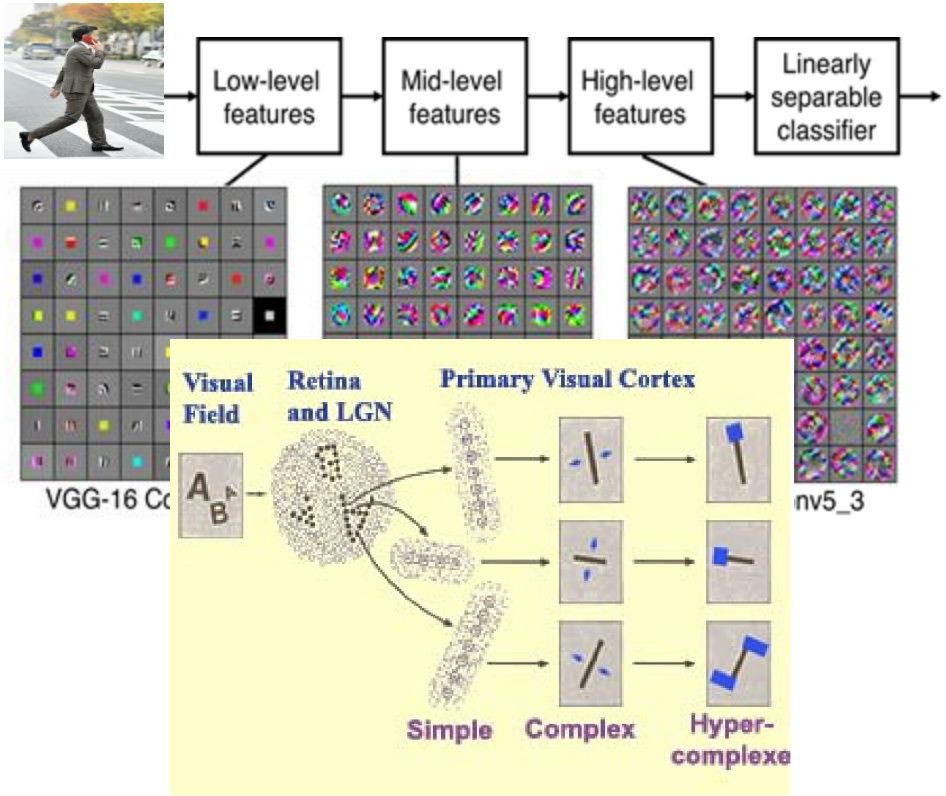




[Zeiler and Fergus 2013]

A. Alahi

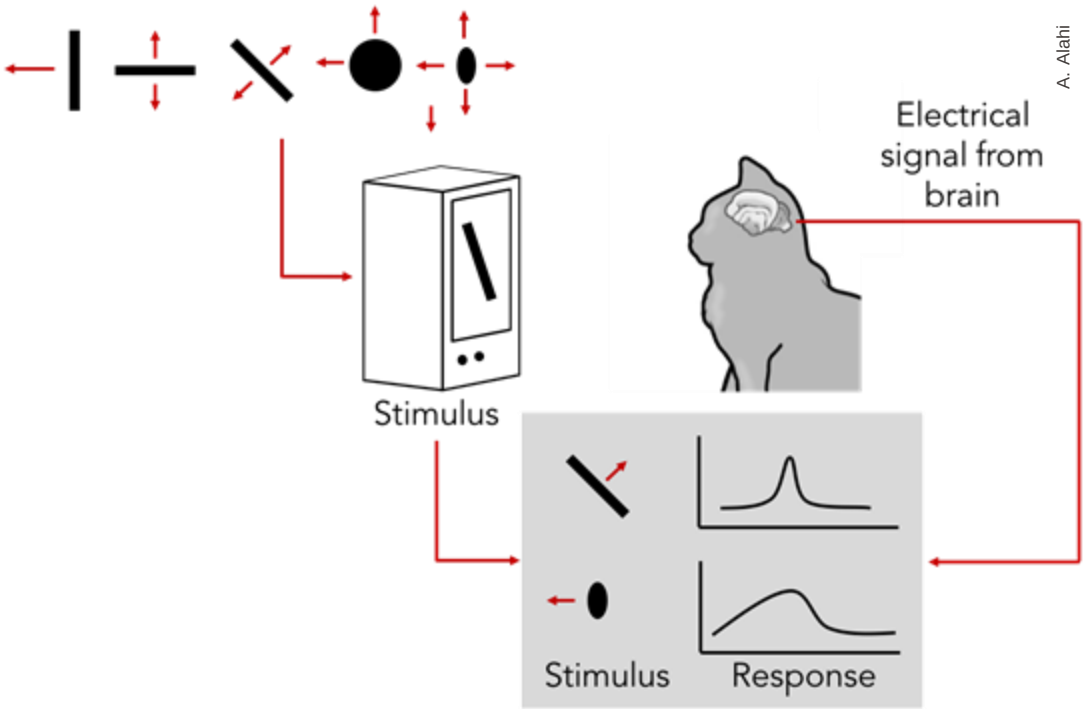
Analogy



A. Alahi

A bit of history

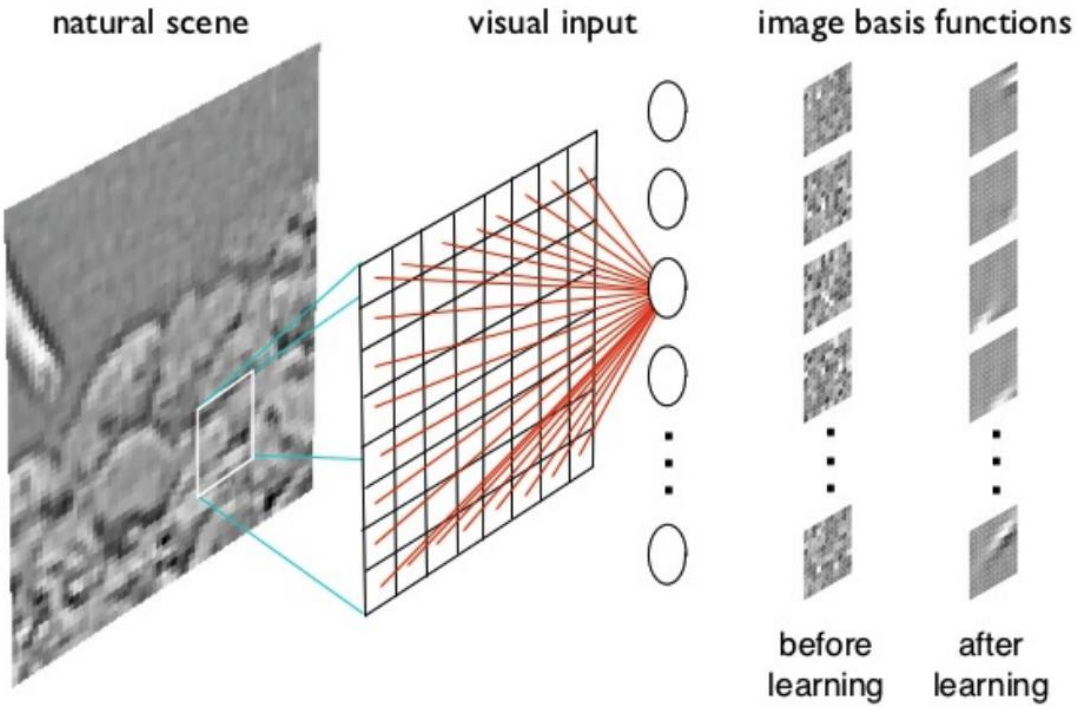
- **Hubel & Wiesel**,
- 1959
  - RECEPTIVE FIELDS OF SINGLE NEURONES IN
  - THE CAT'S STRIATE CORTEX
- 1962
  - RECEPTIVE FIELDS, BINOCULAR INTERACTION
  - AND FUNCTIONAL ARCHITECTURE IN
  - THE CAT'S VISUAL CORTEX
- 1968…



A. Alahi

Cat image by CNX OpenStax is licensed under CC BY 4.0; changes made

Olshausen and Field (1996)



A. Alahi

nature

Explore Content · Journal Information · Publish With Us

nature > letters > article

Published: 13 June 1996

Emergence of simple-cell receptive field properties by learning a sparse code for natural images

Bruno A. Olshausen & David J. Field

Nature **381**, 607–609(1996) | Cite this article

8221 Accesses | **3076** Citations | **0** Altmetric | Metrics

Abstract

THE receptive fields of simple cells in mammalian primary visual cortex can be characterized as being spatially localized, oriented[1–4] and bandpass (selective to structure at different spatial scales), comparable to the basis functions of wavelet transforms[5,6]. One approach to understanding such response properties of visual neurons has been to consider

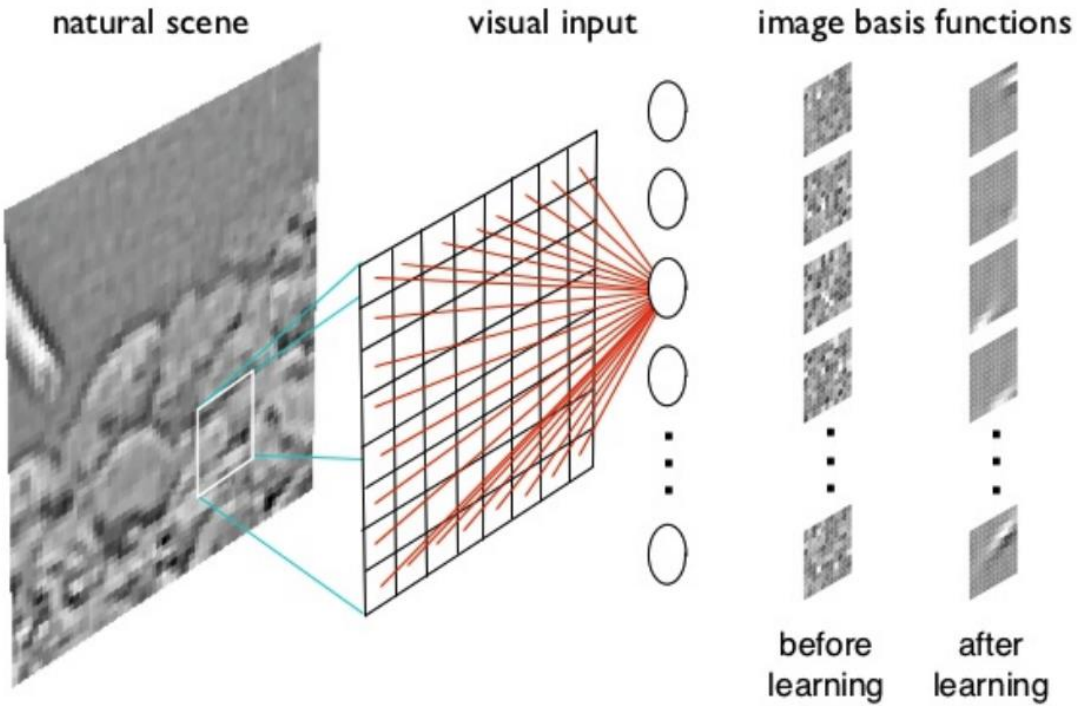
Shahab Bakhtiari @ShahabBakht · Feb 23
Imo, one of the most influential works in computational (vision) neuroscience.

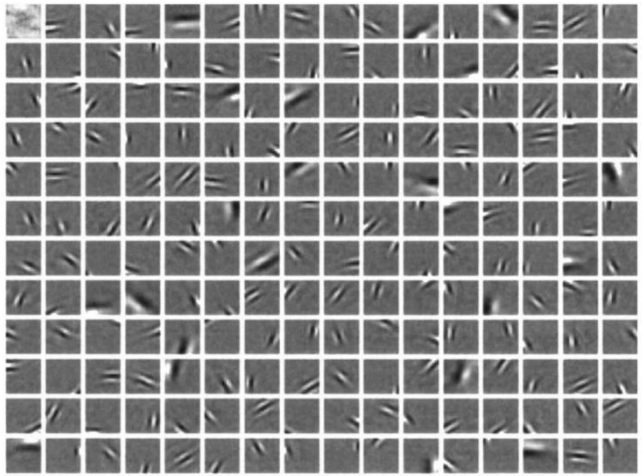
Christopher Rozell @crozSciTech · Feb 22
A paper that changed the trajectory of my career (and my life) when I read it. Written by one of my intellectual heros.

Pierre Vandergheynst @trekkinglemon · Feb 22
20 years ago, I fell in love with that paper. It came straight after I had discovered Marr's book, and it was enthralling. A spell that has not been broken.

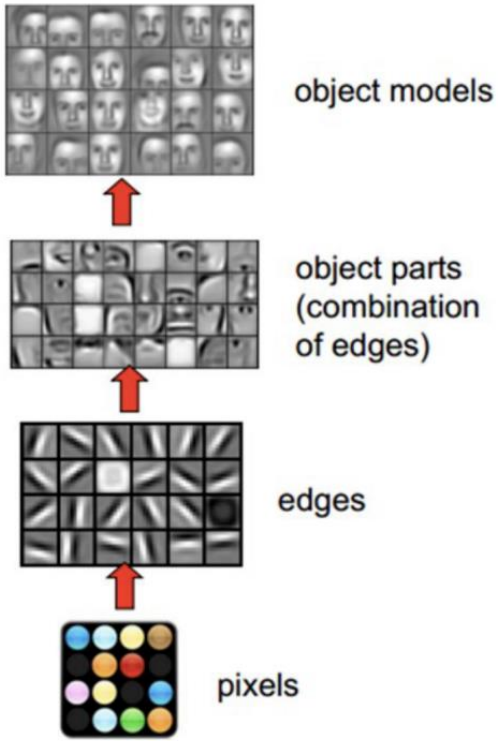
DI

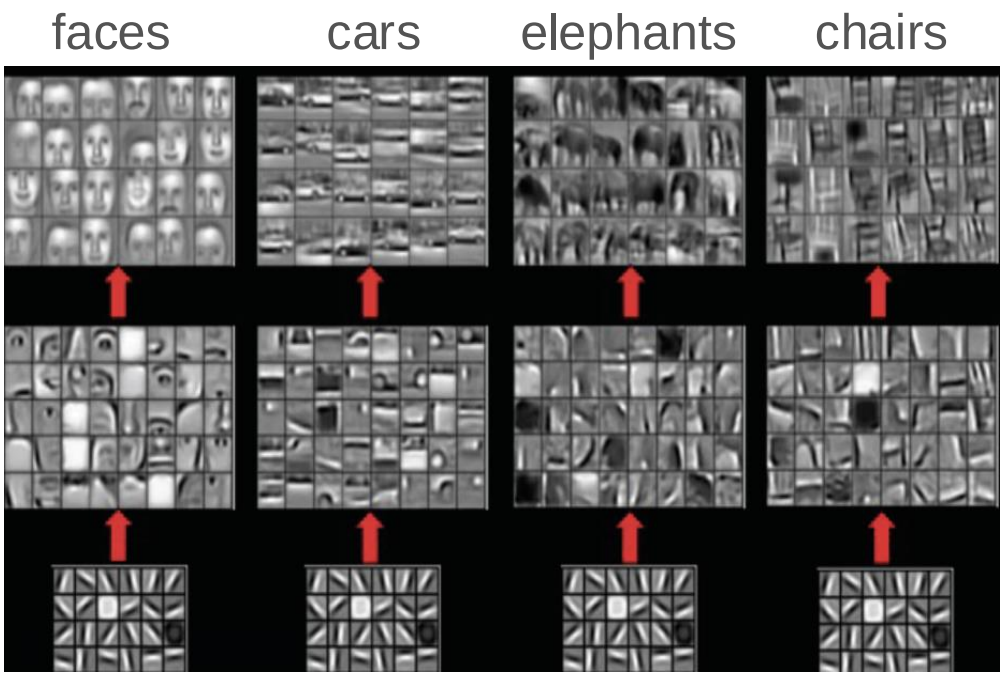
Olshausen and Field (1996)





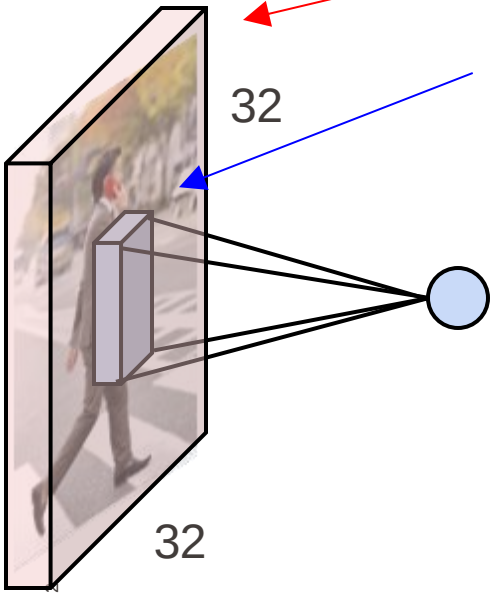
CNN
- Intuitions -





A. Alahi

The brain/neuron view of CONV Layer



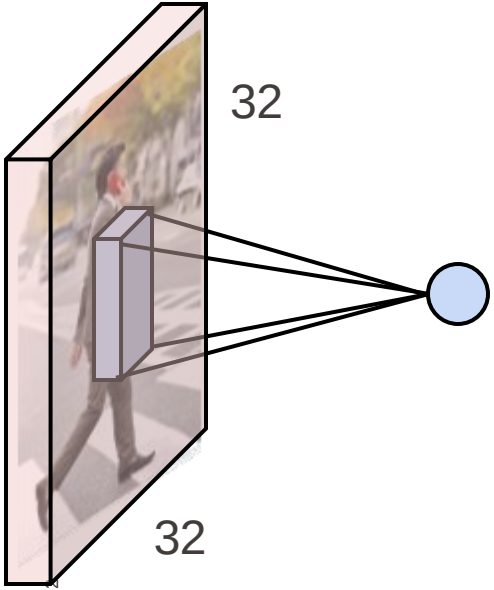
32x32x3 image

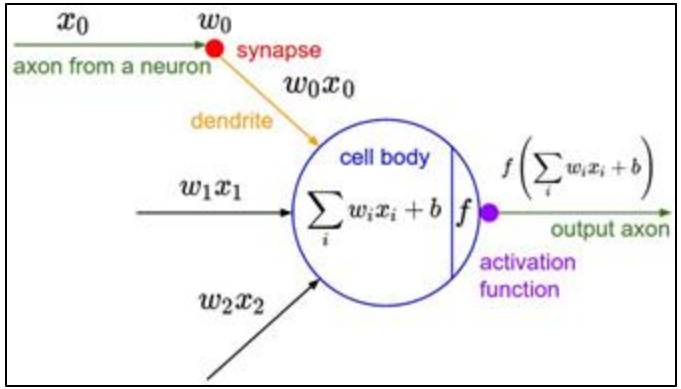
5x5x3 filter

1 number:
the result of taking a dot product between the filter and this part of the image
(i.e., 5*5*3 = 75-dimensional dot product)

A. Alahi

The brain/neuron view of CONV Layer



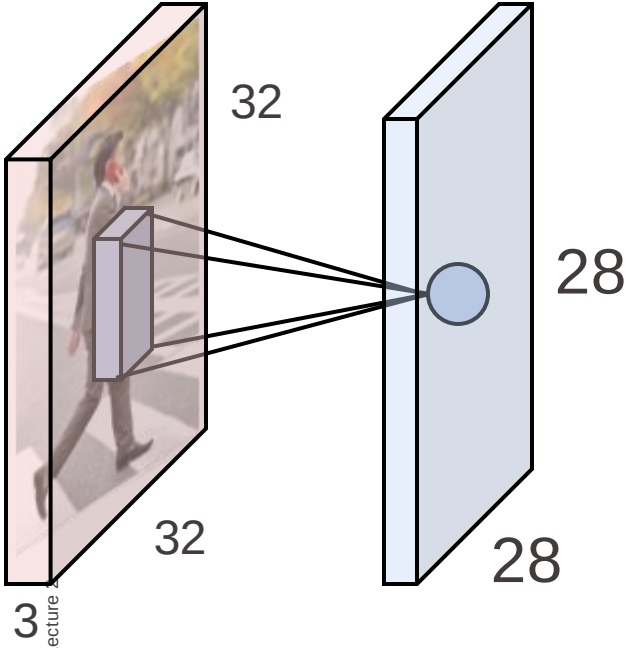


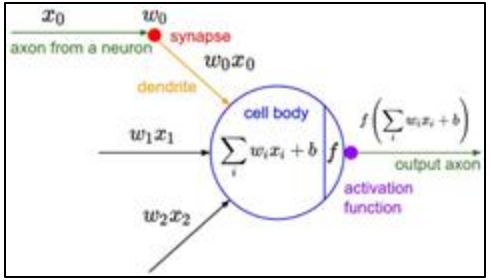
It's just a neuron with local connectivity...

1 number:
the result of taking a dot product between the filter and this part of the image
(i.e., 5*5*3 = 75-dimensional dot product)

A. Alahi

The brain/neuron view of CONV Layer





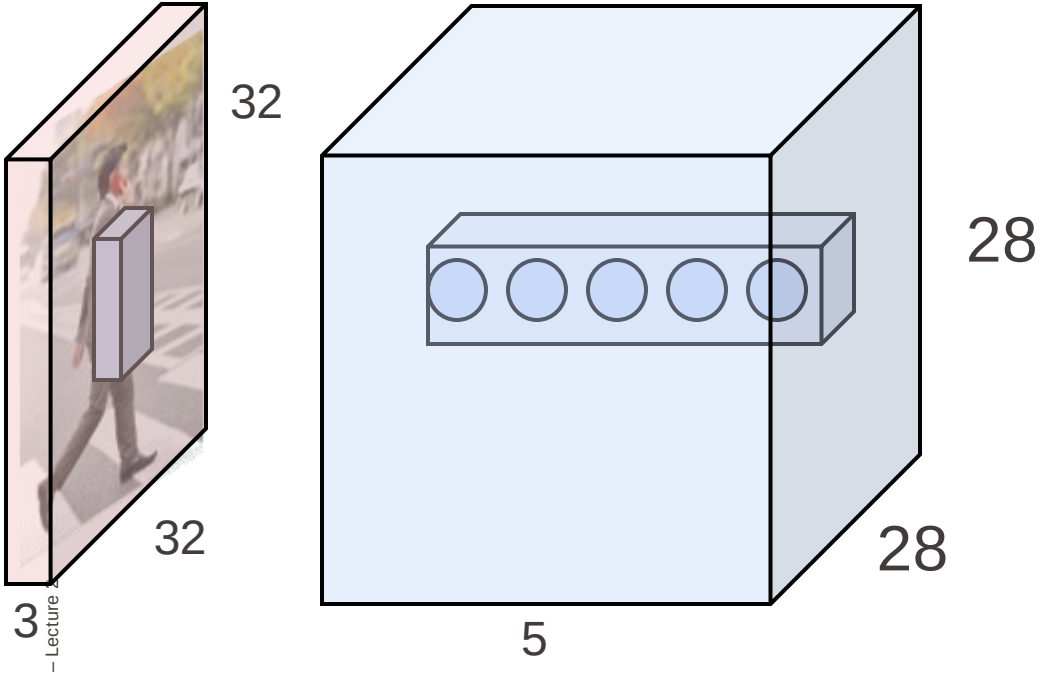
An activation map is a 28x28 sheet of neuron outputs:

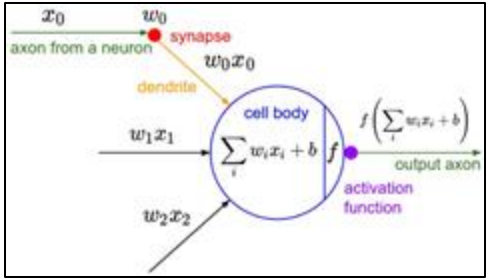
1. Each is connected to a small region in the input
2. All of them share parameters

"5x5 filter" -> "5x5 receptive field for each neuron"

A. Alahi

The brain/neuron view of CONV Layer





A. Alahi

E.g., with 5 filters,
CONV layer consists of neurons arranged in a 3D grid (28x28x5)

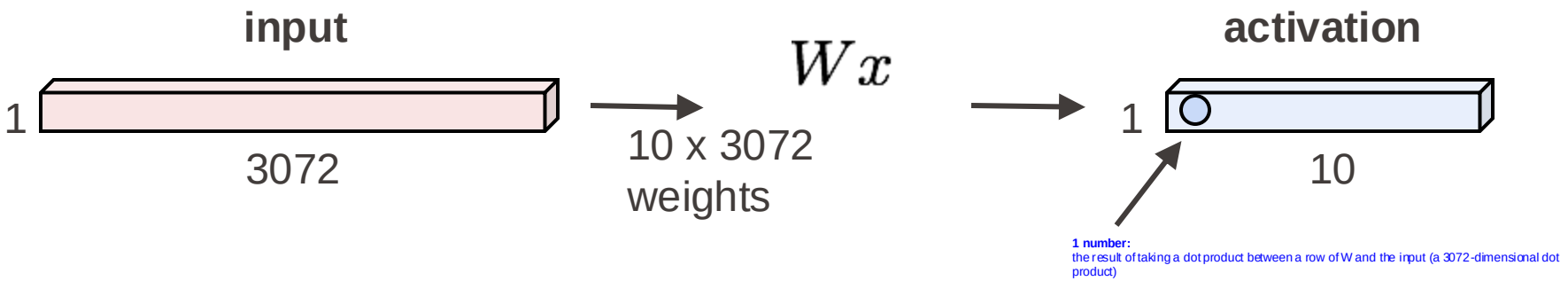
There will be 5 different neurons all looking at the same region in the input volume

Fully Connected Layer

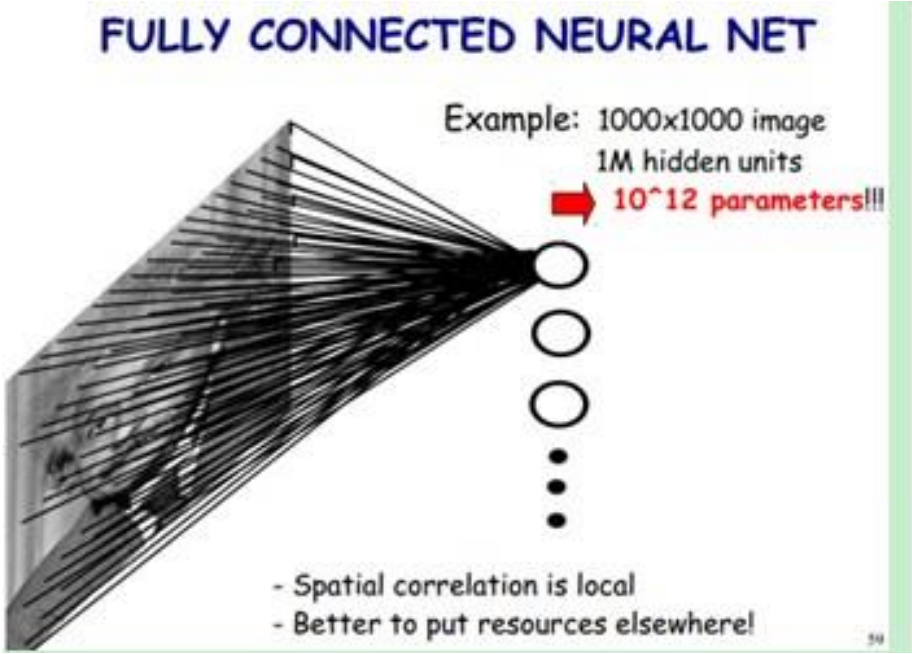
A. Alahi

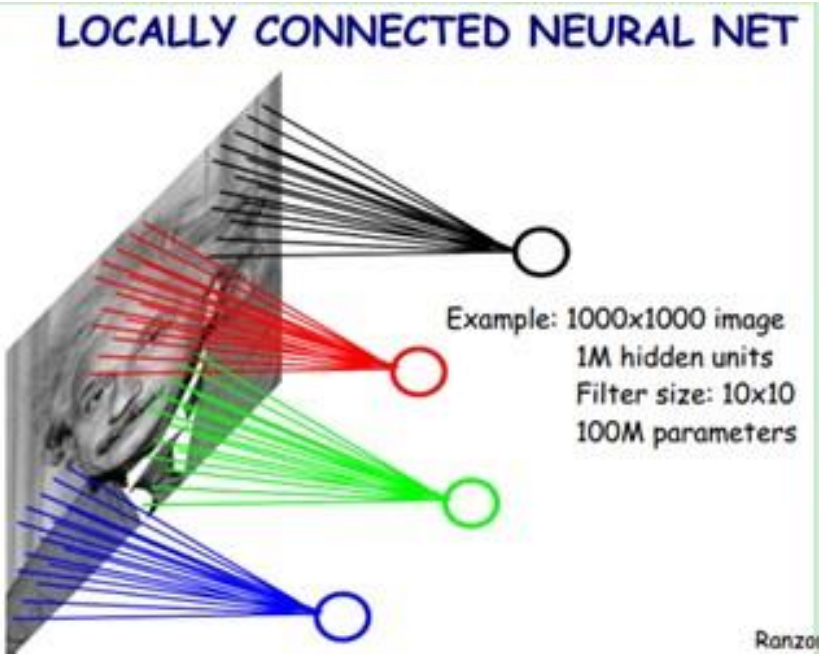
32x32x3 image -> stretch to 3072 x 1

Each neuron looks at the full input volume



Fully connected VS convolution



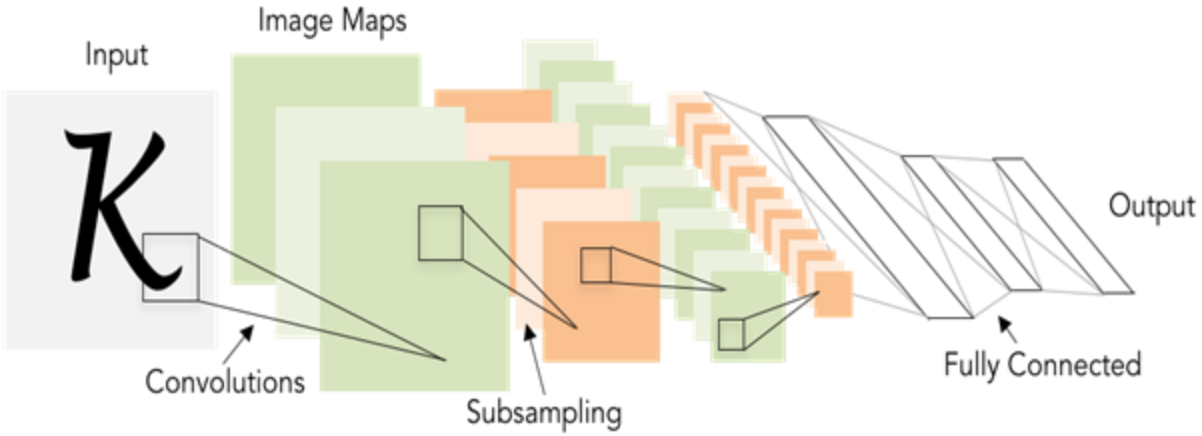


A. Alahi

Popular architectures

A. Alahi

LeNet-5

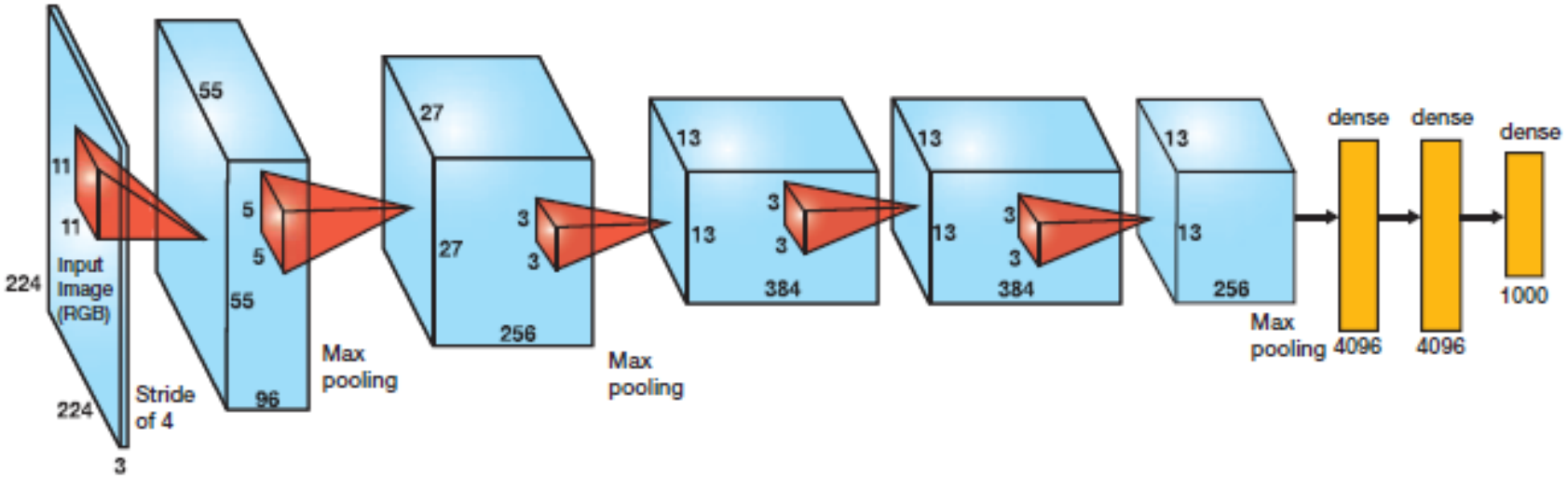


[LeCun, Bottou, Bengio, Haffner 1998]

Popular architectures

A. Alahi

AlexNet

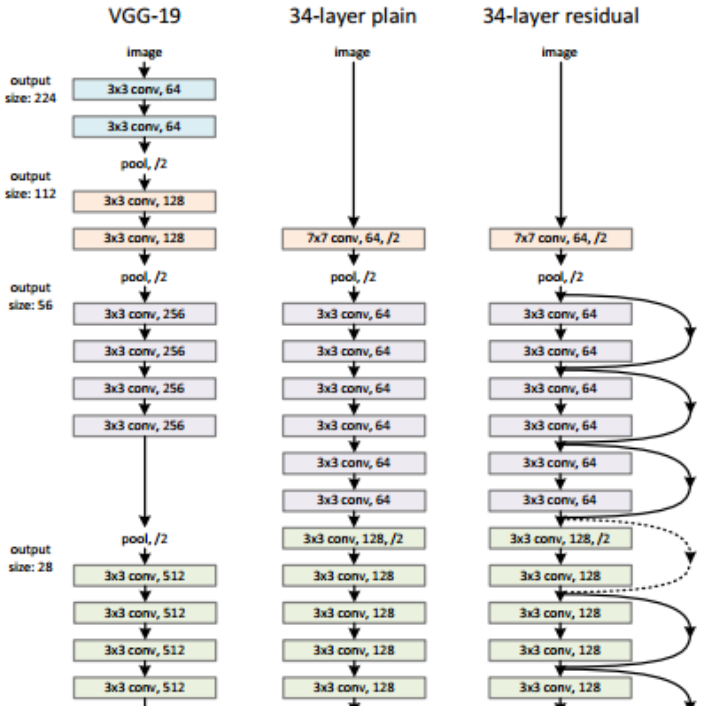


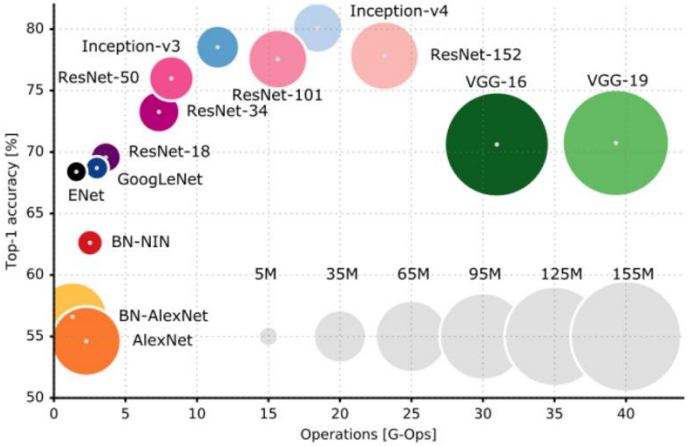
[Krizhevsky, Sutskever, Hinton, 2012]

Popular architectures

A. Alahi

- LeNet
- AlexNet
- VGG
- ResNet
- …

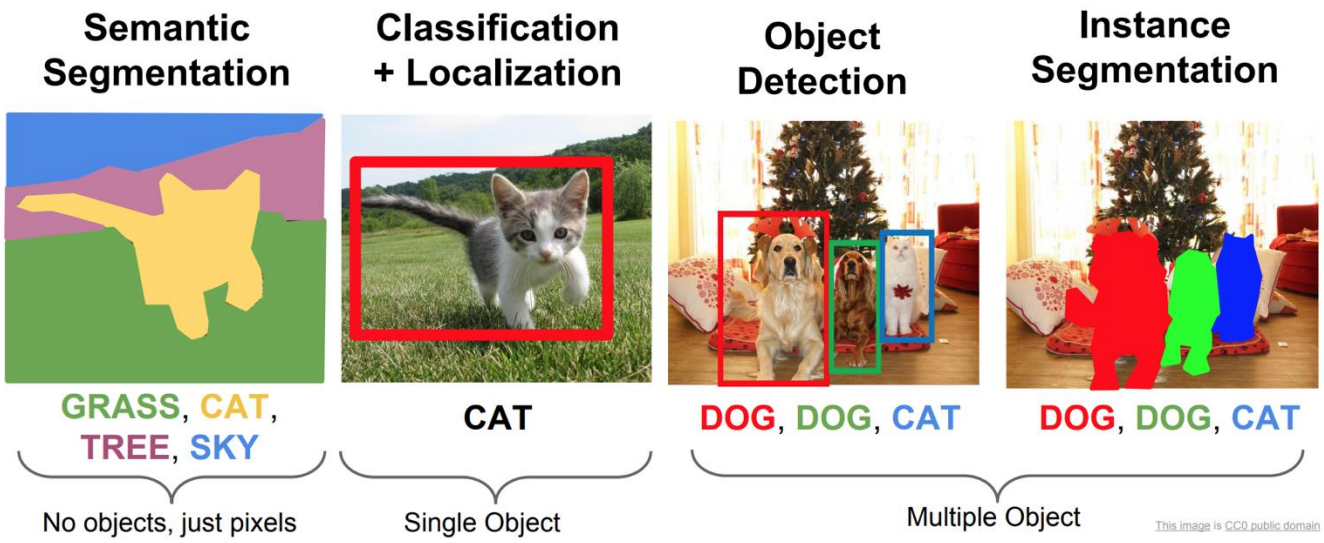




More in the remaining of the class!

Applications of CNN

A. Alahi



More in the remaining of the class!

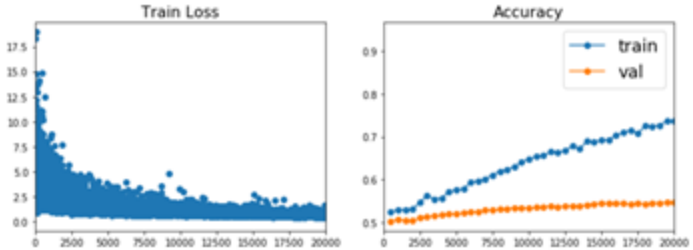
Applications of CNN

Art!

A. Alahi

So far…

- **Convolutional Neural Networks**
- **The challenges to design the architecture (hyper-parameters, # layers, filter size…)**



What can we do to improve the performance of an architecture (i.e., reduce generalization error)?

Regularization

A. Alahi

A. Alahi

Reminder: Linear Classifier

$$f(x_i, W) = Wx_i + b$$

Image x_i

→ $f(\mathbf{x_i}, \mathbf{W})$ → **10** scores → L_i Loss (a scalar)

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1)$$

Suppose that we found a W such that L = 0.
Is this W unique?

No! 2W is also leading to L = 0!

Suppose

3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:

Ped	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Losses:	2.9	0	

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Before:

= max(0, 1.3 - 4.9 + 1)
+max(0, 2.0 - 4.9 + 1)
= max(0, -2.6) + max(0, -1.9)
= 0 + 0
= 0

With W twice as large:

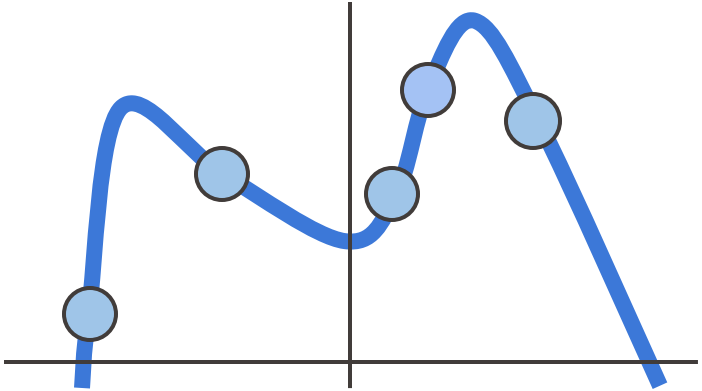
= max(0, 2.6 - 9.8 + 1)
+max(0, 4.0 - 9.8 + 1)
= max(0, -6.2) + max(0, -4.8)
= 0 + 0
= 0

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}$$

Data loss: Model predictions should match training data

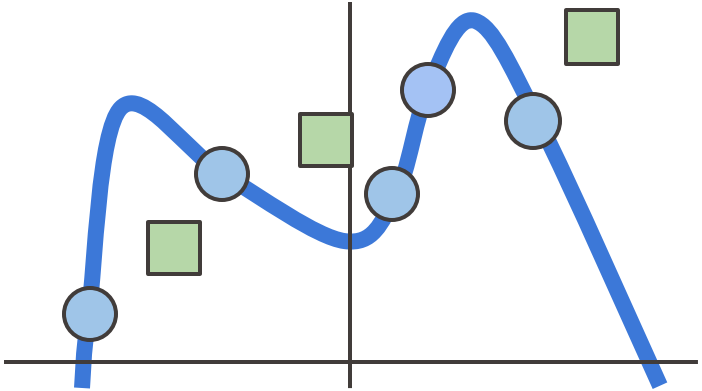
$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}$$

Data loss: Model predictions should match training data



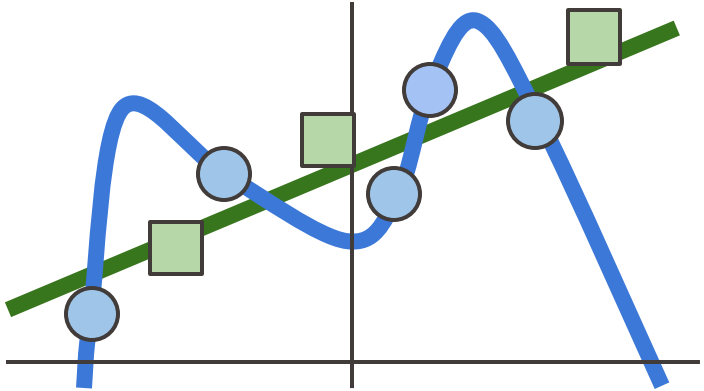
$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}$$

Data loss: Model predictions should match training data



$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}$$

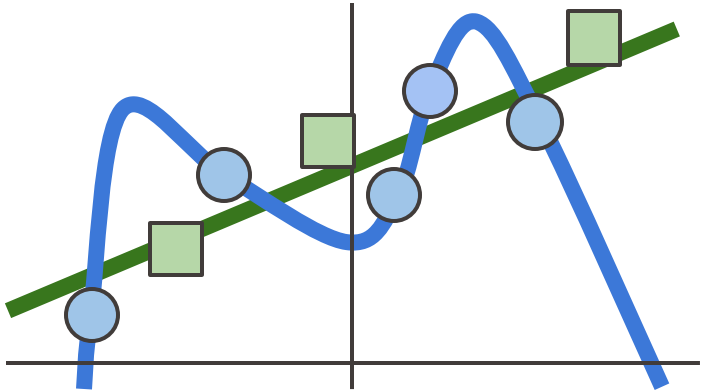
Data loss: Model predictions should match training data



$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)} + \underbrace{\lambda R(W)}$$

Data loss: Model predictions should match training data

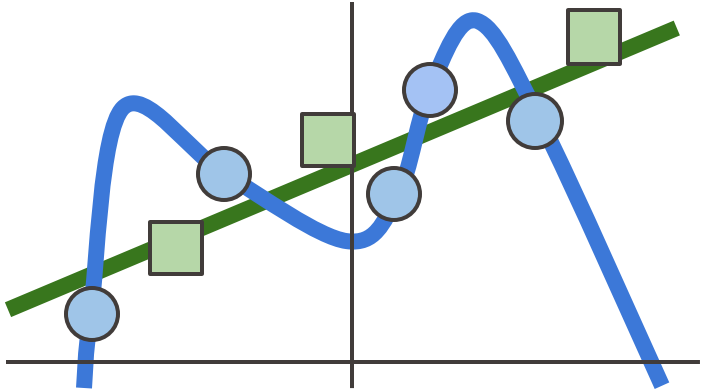
Regularization: Model should be “simple”, so it works on test data



$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)} + \underbrace{\lambda R(W)}$$

Data loss: Model predictions should match training data

Regularization: Model should be “simple”, so it works on test data



Occam’s Razor:
“Among competing hypotheses, the simplest is the best”
William of Ockham, 1285 - 1347

Regularization

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

Where ` λ = regularization strength (hyperparameter)

In common use:

L2 regularization $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2) $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

A. Alahi

L2 Regularization (Weight Decay)

$$x = [1, 1, 1, 1]$$

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

$$w_1^T x = w_2^T x = 1$$

L2 Regularization (Weight Decay)

$$x = [1, 1, 1, 1]$$

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

(If you are a Bayesian: L2 regularization also corresponds MAP inference using a Gaussian prior on W)

$$w_1^T x = w_2^T x = 1$$

Regularization

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

Where λ = regularization strength (hyperparameter)

In common use:

L2 regularization $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2) $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

=> Regularization = simpler models, constraint/penalties on parameters

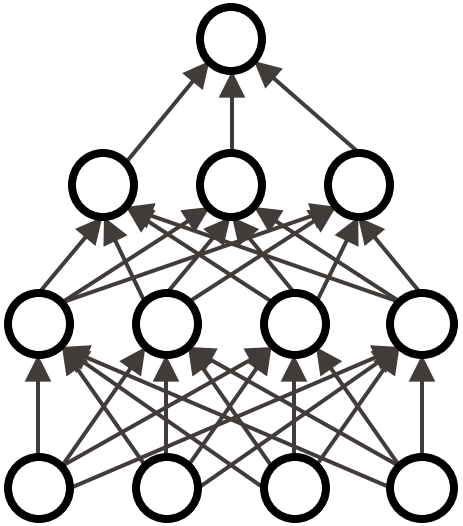
What else?

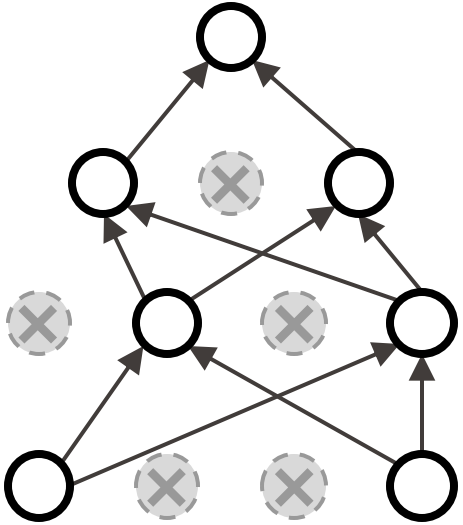
Dropout

Regularization: Dropout

A. Alahi

- In each forward pass, randomly set some neurons to zero
- Probability of dropping is a hyperparameter; 0.5 is common



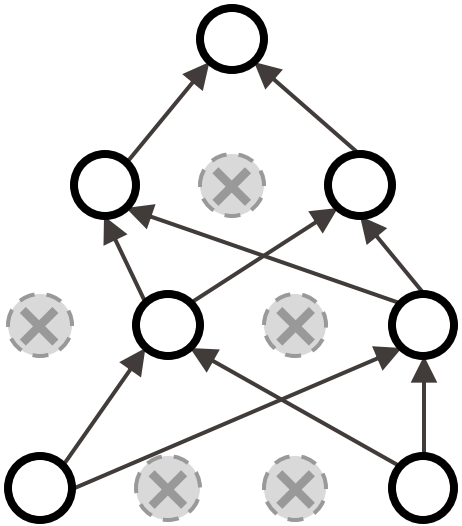


Srivastava et al, “Dropout: A simple way to prevent neural networks from overfitting”, JMLR 2014

Regularization: Dropout

Example forward pass with a 3-layer network using dropout

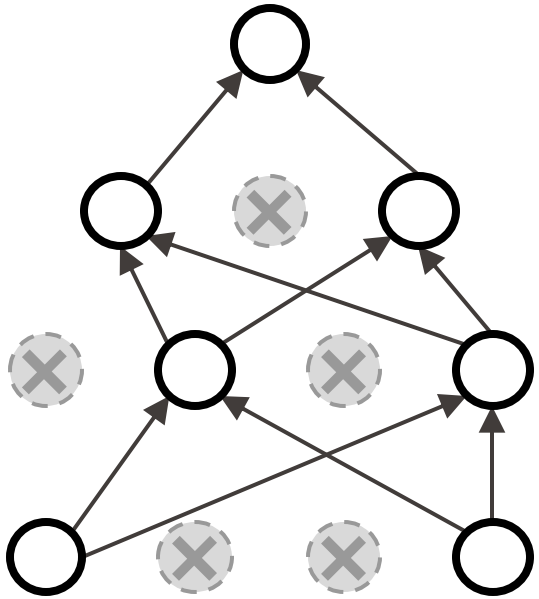
```
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)
```



A. Alahi

Regularization: Dropout

How can this possibly be a good idea?



- One interpretation:
- Dropout is training a large **ensemble** of models (that share parameters).
- Each binary mask is one model
- An FC layer with 4096 units has
- $2^{4096} \sim 10^{1233}$ possible masks!
- Only $\sim 10^{82}$ atoms in the universe...

A. Alahi

Regularization

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

Where λ = regularization strength (hyperparameter)

In common use:

L2 regularization $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2) $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

Dropout

What else?

DATA augmentation

Regularization: Data augmentation

Additional data

Original data

flip

Change illumination

...

A. Alahi

Regularization

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

Where λ = regularization strength (hyperparameter)

In common use:

L2 regularization $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2) $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

Dropout

Data augmentation

What else?

Regularization

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i;W)_j - f(x_i;W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

Where λ = regularization strength (hyperparameter)

In common use:

L2 regularization $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2) $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

Dropout

Data augmentation

Multiple tasks learning

…

References

- Book: Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville
  (http://www.deeplearningbook.org/)

- Class on CNN: http://cs231n.stanford.edu/

- Good tuto on gradient check:

http://cs231n.github.io/neural-networks-3/

- On gradient based optimization methods:

http://ruder.io/optimizing-gradient-descent/

References

- Book: Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville
  (http://www.deeplearningbook.org/)

- Class on CNN: http://cs231n.stanford.edu/

- A few useful things to know about ML by Pedro Domingos

https://homes.cs.washington.edu/~pedrod/papers/cacm12.pdf

- Python Numpy tutorial

http://cs231n.github.io/python-numpy-tutorial/

Additional materials

- Linear classify demo:

http://vision.stanford.edu/teaching/cs231n-demos/linear-classify/

- A visual introduction to machine learning:

  http://www.r2d3.us/visual-intro-to-machine-learning-part-1/

- Nice interactive visualization on some Machine Learning techniques:

  https://distill.pub/